

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

TEMA: PRÁCTICA EXPERIMENTAL UNIDAD III

AquaGest – Sistema para la Gestión Inteligente de una
Cameronera

ESTUDIANTES:

[Nombre Apellido 1]

[Nombre Apellido 2]

[Nombre Apellido 3]

[Nombre Apellido 4]

DOCENTE:

Ing. Guerrero Ulloa Gleiston Ciceron

MATERIA:

Ingeniería de Requerimientos

CURSO:

4to “A” Software

LINK DE GITHUB:

[Colocar enlace de GitHub]

FECHA:

6 de julio de 2026

Índice

1. Datos generales y control del documento	1
1.1. Conformación del equipo e identificación de roles	1
1.2. Historial de versiones	1
1.3. Nombre del archivo de entrega	1
2. Introducción	2
2.1. Descripción breve del sistema AquaGest	2
2.2. Propósito del documento	2
2.3. Alcance de la práctica experimental	3
2.4. Base de trabajo utilizada	3
2.5. Visión general del documento	4
3. Fundamento teórico	4
3.1. Marco normativo aplicado a la especificación de requisitos	4
3.2. Documentación de requisitos	5
3.3. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018	6
3.4. Modelos de datos: perspectiva de datos	7
3.4.1. Modelo entidad-relación	7
3.4.2. Notación Crow's Foot	8
3.4.3. Normalización y tercera forma normal	9
3.5. Modelos funcionales: perspectiva funcional	9
3.5.1. Diagramas de flujo de datos	10
3.5.2. DFD Nivel 0, Nivel 1 y DFD esencial	10
3.5.3. Diagrama de actividades UML	11
3.6. Modelos de comportamiento: perspectiva de comportamiento	11
3.6.1. Máquina de estados UML	12
3.6.2. Diagrama de secuencia UML	12
3.7. Interrelaciones entre las tres perspectivas	13

4. Revisión y refinamiento del SRS parcial	14
4.1. Criterios de revisión aplicados al SRS	14
4.2. Registro de defectos identificados	15
4.3. Requisitos corregidos con sintaxis EARS	16
4.4. Tabla de atributos completa de RF y RNF	18
5. Modelo de datos: ER y clases UML	21
5.1. Identificación de entidades del dominio	21
5.2. Relaciones, cardinalidades y participación	23
5.3. Verificación de tercera forma normal	25
5.4. Diagrama ER en notación Crow's Foot	26
5.5. Transformación del ER al Diagrama de Clases UML	27
5.6. Diagrama de Clases UML	28
6. Modelo funcional: DFD y actividades UML	30
6.1. DFD Nivel 0: diagrama de contexto	30
6.2. DFD Nivel 1	31
6.3. DFD esencial del proceso principal	33
6.4. Balanceo entre DFD Nivel 0 y DFD Nivel 1	34
6.5. Diagrama de Actividades UML del caso de uso principal	35
7. Modelo de comportamiento: estados y secuencia	36
7.1. Entidad seleccionada para el modelo de estados	36
7.2. Identificación de estados, eventos y acciones	37
7.3. Diagrama de Máquina de Estados UML	39
7.4. Diagrama de Secuencia UML del escenario principal	41
8. Matriz de trazabilidad y coherencia entre modelos	42
8.1. Matriz RF por modelos	42
8.2. Requisitos sin cobertura y modelos sin requisito asociado	46
8.3. Verificaciones cruzadas entre modelos	47

8.4. Tabla comparativa de perspectivas de modelado	48
9. Conclusiones	50
9.1. Conclusión 1	50
9.2. Conclusión 2	50
9.3. Conclusión 3	50
9.4. Conclusión 4	50
9.5. Conclusión 5	50
10. Referencias bibliográficas	50
A. Tabla completa de atributos de RF y RNF	52
B. Registro de defectos del SRS con correcciones aplicadas	52
C. Exportaciones originales de diagramas en alta resolución	52
C.1. Diagrama ER	52
C.2. Diagrama de Clases UML	52
C.3. DFD Nivel 0	52
C.4. DFD Nivel 1	52
C.5. DFD esencial	52
C.6. Diagrama de Actividades UML	52
C.7. Máquina de Estados UML	52
C.8. Diagrama de Secuencia UML	52
D. Referencias IEEE y declaración de uso de IA	52
D.1. Captura o evidencia del gestor bibliográfico	52
D.2. Declaración de uso de herramientas de IA	52
E. Declaración individual de aporte de cada integrante	52
E.1. Integrante 1	52
E.2. Integrante 2	52
E.3. Integrante 3	52

E.4. Integrante 4	52
-----------------------------	----

Índice de figuras

1.	Diagrama ER del sistema AquaGest en notación Crow's Foot.	26
2.	Diagrama de Clases UML del sistema AquaGest.	29
3.	DFD Nivel 0: diagrama de contexto del sistema AquaGest.	31
4.	DFD Nivel 1 del sistema AquaGest.	32
5.	DFD esencial del proceso principal del sistema AquaGest.	34
6.	Diagrama de Actividades UML del caso de uso principal.	36
7.	Máquina de Estados UML de la entidad Estanque.	41
8.	Diagrama de Secuencia UML del escenario principal.	42

Índice de tablas

1.	Integrantes del equipo, datos y roles.	1
2.	Historial de versiones del documento.	1
3.	Control del archivo entregable.	1
4.	Comparación entre las perspectivas de modelado aplicadas a AquaGest .	14
5.	Registro de defectos del SRS con corrección propuesta.	15
6.	Requisitos refinados mediante sintaxis EARS.	17
7.	Tabla de atributos de requisitos funcionales.	19
8.	Tabla de atributos de requisitos no funcionales.	19
9.	Entidades del modelo de datos.	21
10.	Relaciones del modelo ER.	23
11.	Verificación de normalización del modelo de datos.	25
12.	Transformación de entidades ER a clases UML.	27
13.	Flujos de datos del DFD Nivel 0.	30
14.	Subprocesos y almacenes internos del DFD Nivel 1.	32
15.	DFD esencial del proceso de negocio principal.	33
16.	Verificación de balanceo entre DFD Nivel 0 y DFD Nivel 1.	34
17.	Comparación de entidades candidatas para el modelo de estados.	37
18.	Estados identificados para la entidad Estanque.	38
19.	Eventos, condiciones y acciones de la entidad Estanque.	38
20.	Sintaxis de transiciones de la máquina de estados.	40
21.	Elementos del diagrama de secuencia del CU-02.	41
22.	Matriz de trazabilidad RF por modelos.	44
23.	Resumen de cobertura de modelos.	46
24.	Gaps identificados en la trazabilidad de modelos.	46
25.	Elementos identificados como posible gold plating.	47
26.	Verificaciones cruzadas entre perspectivas de modelado.	48
27.	Comparación de modelos estructurales, funcionales y de comportamiento.	49

1. Datos generales y control del documento

1.1. Conformación del equipo e identificación de roles

Tabla 1: Integrantes del equipo, datos y roles.

#	Nombres y apellidos	Cédula	Correo institucional	Rol
1				
2				
3				
4				

1.2. Historial de versiones

Tabla 2: Historial de versiones del documento.

Versión	Fecha	Autor	Descripción del cambio
1.0			

1.3. Nombre del archivo de entrega

Tabla 3: Control del archivo entregable.

Elemento	Detalle
Nombre del archivo PDF	PE3_U3_APELLIDO1_APELLIDO2.pdf
Repositorio GitHub	
Fecha de entrega	

2. Introducción

2.1. Descripción breve del sistema AquaGest

AquaGest es un sistema de información para la gestión inteligente de una camaronera, diseñado para centralizar y mejorar el control operativo de los procesos productivos vinculados al cultivo de camarón. El sistema permite gestionar información relacionada con estanques o piscinas, ciclos de siembra, parámetros de calidad del agua, alimentación, historial sanitario, empleados, inventario de insumos, mantenimiento, cosechas, reportes operativos y apoyo mediante componentes de inteligencia artificial.

El sistema está orientado a usuarios como administradores, técnicos acuícolas, operarios de producción, bodegueros, supervisores de mantenimiento y otros actores vinculados a la operación camaronera. Su propósito principal es reducir la dependencia de registros manuales dispersos, mejorar la trazabilidad de la información productiva y apoyar la toma de decisiones basada en datos.

Dentro del alcance funcional del sistema se consideran módulos para registrar estanques, controlar siembras, monitorear parámetros del agua, calcular y registrar alimentación diaria, gestionar inventario de bodega, registrar cosechas y generar reportes comparativos. Asimismo, el proyecto contempla restricciones técnicas asociadas al uso de aplicación web, base de datos relacional, control de acceso por roles y operación con conectividad limitada en campo.

2.2. Propósito del documento

El propósito de este documento es presentar la Práctica Experimental N.º 3 mediante una estructura técnica que integre la revisión del SRS parcial de AquaGest con los modelos requeridos para la especificación de requisitos. Para ello, se documentan los artefactos necesarios para comprobar la coherencia entre los requisitos funcionales y no funcionales, los modelos de datos, los modelos funcionales y los modelos de comportamiento.

De manera específica, el documento busca cumplir con los criterios establecidos para la PE3: revisión y refinamiento del SRS parcial, identificación de defectos, corrección de requisitos mediante EARS, tabla completa de atributos RF/RNF, diagrama entidad-relación, diagrama de clases UML, DFD Nivel 0, DFD Nivel 1, DFD esencial, diagrama de actividades, máquina de estados UML, diagrama de secuencia y matriz de trazabilidad entre modelos.

Además, este documento servirá como evidencia académica del proceso de análisis realizado sobre el sistema AquaGest, permitiendo demostrar que los modelos construidos

no son elementos aislados, sino representaciones derivadas de requisitos previamente identificados, clasificados, priorizados y verificados.

2.3. Alcance de la práctica experimental

El alcance de esta práctica experimental comprende el refinamiento del SRS parcial de AquaGest y la construcción de modelos de especificación de requisitos en herramienta CASE. La práctica se limita al análisis y modelado del sistema, por lo que no incluye implementación de software, codificación de módulos, conexión real con sensores IoT, entrenamiento de modelos de inteligencia artificial ni despliegue funcional de la plataforma.

En primer lugar, se revisan los requisitos funcionales y no funcionales existentes para detectar defectos de calidad, tales como ambigüedad, falta de verificabilidad, ausencia de fuente, duplicidad, inconsistencia o formulaciones incompletas. Posteriormente, se corrigen los requisitos seleccionados mediante sintaxis EARS y se completa la tabla de atributos requerida para RF y RNF.

En segundo lugar, se desarrolla la perspectiva de datos mediante un diagrama ER en notación Crow's Foot y su transformación hacia un diagrama de clases UML. En tercer lugar, se representa la perspectiva funcional mediante DFD Nivel 0, DFD Nivel 1, DFD esencial y diagrama de actividades UML. Finalmente, se modela la perspectiva de comportamiento mediante una máquina de estados UML y un diagrama de secuencia del escenario principal seleccionado.

Por tanto, el alcance de la práctica se concentra en evidenciar la relación entre requisitos y modelos, verificando que cada artefacto construido responda a necesidades reales del sistema AquaGest y no constituya una función adicional sin respaldo en los requisitos.

2.4. Base de trabajo utilizada

La base de trabajo utilizada corresponde al SRS parcial de AquaGest desarrollado en la Entrega 1B del Proyecto Final de Curso. Dicho documento contiene la descripción general del sistema, requisitos funcionales, requisitos no funcionales, restricciones de diseño, interfaces externas, casos de uso, modelo conceptual de clases, priorización MoSCoW y matriz de trazabilidad parcial.

Asimismo, se emplea como referencia la guía de la Práctica Experimental N.º 3, la cual establece que el equipo debe utilizar la lista de requisitos depurada de la PE2 y el SRS parcial de la Entrega 1B como insumos principales. Con base en estos documentos, se desarrollan los modelos solicitados y se verifica que cada uno mantenga relación directa

con los requisitos especificados.

También se consideran los criterios de evaluación de la rúbrica PE3, especialmente aquellos relacionados con la presentación obligatoria de diagramas ER, clases UML, DFD Nivel 0 y Nivel 1, máquina de estados UML, tabla completa de atributos RF/RNF y matriz de trazabilidad entre requisitos y modelos.

2.5. Visión general del documento

El presente documento se organiza de forma progresiva para evidenciar el desarrollo completo de la Práctica Experimental N.º 3. En primer lugar, se presenta la introducción, donde se describe brevemente el sistema AquaGest, el propósito del documento, el alcance de la práctica y la base de trabajo utilizada.

En segundo lugar, se desarrolla el fundamento teórico, compuesto por el marco normativo, la documentación de requisitos, el documento ERS/SRS según IEEE/ISO/IEC 29148:2018, los modelos de datos, los modelos funcionales, los modelos de comportamiento y las interrelaciones entre las tres perspectivas.

En tercer lugar, se presenta la revisión y refinamiento del SRS parcial, incluyendo la tabla de defectos, las correcciones mediante sintaxis EARS y la tabla completa de atributos para requisitos funcionales y no funcionales.

En cuarto lugar, se desarrolla el modelo de datos, compuesto por la identificación de entidades, relaciones, normalización, diagrama ER en notación Crow's Foot y diagrama de clases UML. Luego, se presenta el modelo funcional mediante DFD Nivel 0, DFD Nivel 1, DFD esencial y diagrama de actividades UML.

Posteriormente, se documenta el modelo de comportamiento, que incluye la entidad seleccionada para el análisis de estados, la máquina de estados UML y el diagrama de secuencia del caso de uso principal. Finalmente, se incorpora la matriz de trazabilidad y coherencia entre modelos, la tabla comparativa de perspectivas, las conclusiones y los anexos correspondientes.

3. Fundamento teórico

3.1. Marco normativo aplicado a la especificación de requisitos

La especificación de requisitos de software debe desarrollarse bajo un marco normativo que permita organizar, documentar, validar y verificar las necesidades del sistema. IEEE/ISO/IEC 29148:2018 establece lineamientos para los procesos y productos de la

Ingeniería de Requisitos, incluyendo la estructura de la Software Requirements Specification (SRS), la calidad de los requisitos y la necesidad de mantener trazabilidad entre necesidades, requisitos, diseño y pruebas [1]. Esta norma resulta especialmente relevante para AquaGest, porque el sistema requiere transformar información obtenida mediante entrevista, encuesta y observación en requisitos funcionales verificables, tales como registrar siembras, controlar parámetros del agua, calcular raciones de alimento y generar reportes de producción.

De igual forma, IEEE/ISO/IEC 15288:2023 e IEEE/ISO/IEC 12207:2017 complementan este marco al ubicar los requisitos dentro del ciclo de vida de sistemas y software. IEEE/ISO/IEC 15288:2023 aborda la ingeniería de sistemas desde una perspectiva integral, mientras que IEEE/ISO/IEC 12207:2017 organiza procesos específicos del ciclo de vida del software, incluyendo análisis, diseño, implementación, verificación y validación [2, 3]. En AquaGest, esta visión permite comprender que los requisitos no son elementos aislados, sino insumos que deben mantenerse coherentes con los modelos, la arquitectura, las pruebas y las decisiones de diseño posteriores.

Además, ISO/IEC 25010:2023 proporciona un modelo de calidad para productos de software, útil para clasificar requisitos no funcionales relacionados con desempeño, seguridad, fiabilidad, usabilidad, compatibilidad, mantenibilidad y portabilidad [4]. En el sistema AquaGest, este estándar permite fundamentar requisitos como disponibilidad mensual, tiempo de respuesta para registrar parámetros del agua, control de acceso basado en roles y tolerancia a fallos mediante operación offline. Por tanto, el marco normativo no solo orienta la redacción de requisitos, sino también su evaluación objetiva mediante métricas y criterios de verificación.

3.2. Documentación de requisitos

La documentación de requisitos consiste en registrar de manera estructurada las necesidades, restricciones, reglas de negocio y condiciones de calidad que debe cumplir un sistema [1, 5]. Un requisito correctamente documentado debe ser claro, necesario, verificable, trazable, factible y no ambiguo. Esta condición evita que el equipo de desarrollo implemente funcionalidades incorrectas o incompletas, y permite que los testers diseñen casos de prueba directamente vinculados con cada requisito.

En una ERS/SRS, los requisitos suelen clasificarse en requisitos funcionales, requisitos no funcionales y restricciones. Los requisitos funcionales describen comportamientos observables del sistema, es decir, lo que el sistema debe hacer. Los requisitos no funcionales describen condiciones de calidad bajo las cuales el sistema debe operar, mientras que las restricciones delimitan decisiones obligatorias de diseño, tecnología, normativa o entorno [1, 4]. En AquaGest, por ejemplo, “registrar una siembra” es un requisito funcional; “res-

ponder en menos de dos segundos bajo carga normal” es un requisito no funcional; y “usar una base de datos relacional” es una restricción tecnológica.

Asimismo, la documentación de requisitos debe conservar atributos mínimos que faciliten su control durante el ciclo de vida. Entre estos atributos se incluyen identificador, descripción, tipo, prioridad, fuente, estabilidad, estado y verificabilidad [8, 5]. En AquaGest, estos atributos permiten rastrear que el requisito RF-05, relacionado con el registro de siembra, proviene de evidencias de campo y se vincula con actores como el Operario de Producción y el Administrador. Esta trazabilidad fortalece la validez del requisito, porque demuestra que no fue inventado por el equipo, sino derivado de una necesidad del dominio camaronero.

Por otra parte, la documentación de requisitos también debe reducir ambigüedades mediante patrones de redacción controlada. Una técnica útil es EARS, que propone estructuras como “The system shall...” para requisitos universales o “While [condition], the system shall...” para requisitos dependientes de estado [14]. En el caso de AquaGest, un requisito como “el sistema debe registrar parámetros del agua” puede refinarse como: “The AquaGest system shall record water parameter measurements with timestamp, pond identifier, parameter type and value”. Esta redacción permite verificar con mayor precisión las entradas, salidas y condiciones esperadas.

3.3. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018

El documento ERS/SRS es el artefacto formal que especifica los requisitos de software de un sistema. Según IEEE/ISO/IEC 29148:2018, la SRS debe describir el propósito del producto, su alcance, definiciones, referencias, perspectiva general, funciones principales, características de usuarios, restricciones, supuestos, interfaces externas, requisitos específicos y criterios de verificación [1]. Esta estructura permite que el documento funcione como contrato técnico entre los interesados, el equipo de desarrollo y el equipo de pruebas.

En el proyecto AquaGest, la ERS/SRS cumple una función articuladora, porque integra la información obtenida en las fases de elicitación con los requisitos formalizados y los modelos de análisis. El sistema busca gestionar una camaronera mediante módulos de estanques, siembras, monitoreo del agua, alimentación, sanidad, empleados, inventario, mantenimiento, cosechas, reportes e inteligencia artificial. Por ello, la SRS debe ser suficientemente precisa para que cada módulo pueda desarrollarse, probarse y validarse contra las necesidades de los stakeholders.

Además, una SRS de calidad debe mantener consistencia interna entre requisitos, modelos y restricciones. Si un requisito funcional indica que el sistema debe generar alertas

por parámetros del agua fuera de rango, entonces los modelos deben reflejar entidades como *ParametroAgua* y *AlertaAgua*, procesos como *Registrar parámetros del agua* y comportamientos como la transición de una alerta desde “pendiente” hasta “atendida”. Esta coherencia evita contradicciones entre la especificación textual y los diagramas de análisis [11, 9].

De igual manera, IEEE/ISO/IEC 29148:2018 resalta la importancia de que los requisitos sean verificables [1]. En AquaGest, esto implica que no basta con afirmar que el sistema será “rápido”, “seguro” o “fácil de usar”. Cada requisito no funcional debe expresarse con métricas, condiciones y métodos de verificación. Por ejemplo, el requisito de desempeño debe indicar un tiempo máximo de respuesta, la carga esperada y el método de prueba, como una prueba de carga con usuarios concurrentes.

3.4. Modelos de datos: perspectiva de datos

Los modelos de datos representan la estructura informacional del sistema, incluyendo entidades, atributos, relaciones, restricciones de cardinalidad y reglas de integridad [9]. Esta perspectiva responde a la pregunta: ¿qué información necesita almacenar y relacionar el sistema para cumplir sus requisitos? En sistemas transaccionales como AquaGest, el modelo de datos es esencial porque los procesos de siembra, alimentación, monitoreo, inventario y cosecha dependen de registros persistentes y relacionados entre sí.

En AquaGest, la perspectiva de datos permite identificar entidades relevantes del dominio camaronero, tales como *Estanque*, *Siembra*, *Grameo*, *ParametroAgua*, *AlertaAgua*, *RegistroAlimentacion*, *Insumo*, *MovimientoInventario*, *Empleado*, *Cosecha* y *PrediccionIA*. Estas entidades surgen de los requisitos funcionales y de las operaciones reales de la camaronera. Por ejemplo, el RF-09 exige registrar parámetros del agua; por tanto, el modelo de datos debe incluir una entidad que almacene el tipo de parámetro, valor, unidad, fecha, hora y estanque asociado.

Asimismo, la perspectiva de datos facilita la trazabilidad entre requisitos y almacenamiento. Si el sistema debe generar reportes comparativos entre ciclos productivos, entonces la base de datos debe conservar información histórica de siembras, grameos, alimentación, cosechas y costos. Por esta razón, el modelo de datos no debe diseñarse únicamente como una lista de tablas, sino como una representación lógica del dominio que soporte consultas, validaciones, reportes y futuras integraciones.

3.4.1. Modelo entidad–relación

El modelo entidad–relación permite representar conceptualmente los datos de un sistema mediante entidades, atributos y relaciones [9]. Una entidad representa un objeto

relevante del dominio; un atributo describe una propiedad de esa entidad; y una relación expresa cómo se asocian dos o más entidades. Este modelo es útil en etapas tempranas porque permite validar con los interesados si la estructura informacional del sistema refleja correctamente la realidad del negocio.

En AquaGest, una entidad central es *Estanque*, debido a que varias operaciones productivas se organizan alrededor de cada piscina camaronera. Esta entidad puede contener atributos como *idEstanque*, *nombre*, *superficieHa*, *capacidadLarvas* y *estado*. A partir de ella se relacionan entidades como *Siembra*, *ParametroAgua*, *RegistroAlimentacion* y *Cosecha*. Por ejemplo, un estanque puede tener muchas siembras a lo largo del tiempo, pero cada siembra pertenece a un solo estanque.

Además, el modelo entidad–relación permite detectar reglas de negocio relevantes. En AquaGest, una *Siembra* debe asociarse a un estanque existente y debe registrar información como fecha de inicio, cantidad de larvas, laboratorio de origen y responsable. De igual manera, una *Cosecha* debe asociarse a una siembra o ciclo productivo, porque sus datos de peso, talla, rendimiento y costo dependen de la producción acumulada durante ese ciclo.

3.4.2. Notación Crow's Foot

La notación Crow's Foot es una forma gráfica de representar cardinalidades y participación en un modelo entidad–relación [9]. Esta notación permite indicar si una relación es de uno a uno, uno a muchos o muchos a muchos, así como si la participación de una entidad es obligatoria u opcional. Su uso resulta adecuado para la PE3 porque la guía solicita construir el diagrama ER con esta notación y verificar la coherencia de las relaciones.

En AquaGest, la relación entre *Estanque* y *Siembra* puede representarse como uno a muchos: un estanque puede registrar cero o muchas siembras durante su vida operativa, mientras que cada siembra pertenece a un único estanque. De forma similar, una *Siembra* puede tener muchos *Grameos*, porque el peso promedio del camarón se mide periódicamente durante el ciclo productivo. Estas cardinalidades son necesarias para evitar inconsistencias, como registrar un grameo sin una siembra activa.

También pueden existir relaciones muchos a muchos que requieren tablas o clases asociativas. Por ejemplo, un *Empleado* puede participar en varias *TareasDiarias*, y una tarea puede requerir la participación de varios empleados. Esta relación puede resolverse mediante una entidad asociativa denominada *AsignacionTarea*. De igual manera, una *Cosecha* puede utilizar varios *Insumos*, y un insumo puede estar presente en varias cosechas; esta relación puede resolverse con una entidad asociativa como *DetalleCosto-Cosecha*.

3.4.3. Normalización y tercera forma normal

La normalización es un proceso de diseño de bases de datos relacionales que busca reducir redundancia, evitar anomalías de inserción, actualización y eliminación, y mejorar la integridad de los datos [13]. La primera forma normal exige que los atributos sean atómicos; la segunda forma normal elimina dependencias parciales respecto a claves compuestas; y la tercera forma normal elimina dependencias transitivas entre atributos no clave.

En AquaGest, la normalización permite evitar duplicación innecesaria de información. Por ejemplo, si cada registro de alimentación almacenara repetidamente el nombre, superficie y estado del estanque, se produciría redundancia y riesgo de inconsistencias. En una estructura normalizada, el registro de alimentación solo debería almacenar la clave del estanque y sus propios datos operativos, como fecha, cantidad de alimento, tipo de balanceado y responsable. La información descriptiva del estanque debe mantenerse en la entidad *Estanque*.

Asimismo, la tercera forma normal contribuye a mantener la calidad del modelo de datos. Si una entidad *MovimientoInventario* almacena el nombre del insumo y también su categoría, pero la categoría depende del insumo y no del movimiento, entonces existe una dependencia transitiva. Para corregirlo, la categoría debe pertenecer a la entidad *Insumo*, mientras que *MovimientoInventario* debe registrar la entrada o salida, cantidad, fecha y empleado responsable. Esta separación mejora la consistencia del inventario y facilita los reportes.

3.5. Modelos funcionales: perspectiva funcional

Los modelos funcionales describen las transformaciones que realiza el sistema sobre los datos, mostrando entradas, procesos, salidas, almacenes y actores externos [9]. Esta perspectiva responde a la pregunta: ¿qué hace el sistema con la información que recibe? En la Ingeniería de Requisitos, los modelos funcionales ayudan a comprender el flujo de trabajo, descubrir omisiones, detectar procesos duplicados y validar si los requisitos funcionales están completos.

En AquaGest, la perspectiva funcional permite representar procesos como registrar siembra, registrar parámetros del agua, generar alertas, calcular ración diaria, actualizar inventario, registrar cosecha y generar reportes comparativos. Estos procesos no solo almacenan datos, sino que transforman información operativa en resultados útiles para la toma de decisiones. Por ejemplo, una lectura de oxígeno disuelto puede convertirse en un registro histórico y, si está fuera del rango óptimo, en una alerta que requiere acción correctiva.

Además, los modelos funcionales se relacionan con los requisitos porque cada proceso debe estar respaldado por uno o varios RF. El proceso *Registrar parámetros del agua* se vincula con RF-09, RF-10 y RF-11, mientras que el proceso *Registrar alimentación diaria* se vincula con RF-12, RF-13 y RF-14. Esta relación ayuda a verificar que ningún requisito funcional quede sin representación en los modelos.

3.5.1. Diagramas de flujo de datos

Los diagramas de flujo de datos, conocidos como DFD, representan cómo la información entra, se transforma, se almacena y sale de un sistema [9]. Sus elementos principales son procesos, entidades externas, almacenes de datos y flujos de datos. A diferencia de los diagramas UML orientados a objetos, los DFD se concentran en el movimiento lógico de la información, sin detallar aspectos de interfaz, tecnología o implementación física.

En AquaGest, un DFD permite representar que el *Sensor IoT* o el *Técnico Acuícola* envían lecturas de pH, oxígeno, temperatura, salinidad, turbidez y alcalinidad hacia el proceso *Registrar parámetros del agua*. Luego, el sistema almacena dichas lecturas en el repositorio de parámetros y ejecuta una comparación con rangos configurados. Si el valor está fuera de rango, se genera un flujo de salida denominado *alerta de agua* hacia el técnico responsable.

De igual manera, los DFD ayudan a identificar almacenes internos requeridos por el sistema. Para AquaGest, algunos almacenes necesarios son *D1 Estanques*, *D2 Siembras*, *D3 Parámetros del Agua*, *D4 Alimentación*, *D5 Inventario*, *D6 Cosechas* y *D7 Alertas*. La presencia de estos almacenes permite comprobar la coherencia entre la perspectiva funcional y la perspectiva de datos.

3.5.2. DFD Nivel 0, Nivel 1 y DFD esencial

El DFD Nivel 0, también llamado diagrama de contexto, representa el sistema completo como un único proceso central y muestra sus interacciones con entidades externas [9]. En AquaGest, el proceso central sería *Sistema AquaGest*, conectado con entidades como Administrador, Técnico Acuícola, Operario de Producción, Bodeguero, Sensor IoT y Módulo de IA. Este nivel permite delimitar el alcance del sistema y diferenciar qué pertenece al software y qué pertenece al entorno externo.

El DFD Nivel 1 descompone el proceso central en subprocesos principales. En AquaGest, esta descomposición puede incluir al menos cinco procesos: *Gestionar estanques*, *Controlar siembras*, *Monitorear parámetros del agua*, *Gestionar alimentación*, *Gestionar inventario*, *Registrar cosechas* y *Generar reportes*. Para que el modelo sea correcto, debe existir balanceo entre el Nivel 0 y el Nivel 1, es decir, los flujos que entran y salen del

sistema en el diagrama de contexto deben conservarse en la descomposición.

Por su parte, el DFD esencial representa la lógica del negocio sin enfocarse en detalles tecnológicos o de implementación [9]. En AquaGest, un proceso esencial adecuado sería *Monitorear calidad del agua y generar alerta*, debido a que constituye una función crítica para reducir riesgos de mortalidad y mejorar la toma de decisiones. Este DFD esencial debería mostrar la recepción de mediciones, validación del valor, comparación contra rangos óptimos, almacenamiento del parámetro y generación de alerta cuando corresponda.

3.5.3. Diagrama de actividades UML

El diagrama de actividades UML representa el flujo de acciones, decisiones, bifurcaciones, sincronizaciones y objetos dentro de un proceso [12, 5]. A diferencia del DFD, que enfatiza el flujo de datos, el diagrama de actividades muestra el orden de ejecución de las actividades y permite asignar responsabilidades mediante *swimlanes*. Esta característica es útil para modelar procesos donde interactúan usuarios, sistema y componentes externos.

En AquaGest, un diagrama de actividades puede aplicarse al caso de uso *Registrar alimentación diaria*. El flujo iniciaría cuando el Operario de Producción selecciona el estanque; luego, el sistema verifica si existe una siembra activa y si hay grameo previo. Si los datos existen, calcula la ración sugerida; posteriormente, el operario confirma o ajusta la cantidad suministrada, registra el horario y responsable, y el sistema guarda el consumo. Finalmente, el registro queda disponible para el cálculo del FCR.

Además, este diagrama puede incluir nodos de decisión y flujos alternativos. Por ejemplo, si no existe grameo previo, el sistema puede permitir un registro manual sin sugerencia automática. Si faltan campos obligatorios, el sistema debe impedir guardar el registro. Esta representación permite validar que el flujo funcional coincide con los requisitos RF-12, RF-13 y RF-14.

3.6. Modelos de comportamiento: perspectiva de comportamiento

Los modelos de comportamiento describen cómo el sistema o sus objetos cambian a través del tiempo en respuesta a eventos, condiciones y mensajes [5, 12]. Esta perspectiva responde a la pregunta: ¿cómo reacciona el sistema ante situaciones específicas? En la Ingeniería de Requisitos, los modelos de comportamiento son importantes porque permiten analizar estados, transiciones, secuencias de interacción, escenarios alternativos

y reglas temporales.

En AquaGest, esta perspectiva resulta necesaria porque varias entidades tienen ciclos de vida definidos. Un *Estanque* puede pasar por estados como preparación, activo, en cosecha, mantenimiento e inactivo. Una *AlertaAgua* puede pasar por estados como generada, pendiente, atendida y cerrada. Una *OrdenMantenimiento* puede transitar entre pendiente, en proceso, concluida o cancelada. Estas transiciones deben estar alineadas con las reglas de negocio y con los requisitos funcionales.

Asimismo, los modelos de comportamiento permiten validar escenarios críticos. Por ejemplo, cuando el sistema recibe una lectura de pH fuera de rango, debe registrar el parámetro, generar una alerta, notificar al técnico, esperar una acción correctiva y cambiar el estado de la alerta cuando sea atendida. Esta secuencia no puede observarse completamente en el modelo de datos ni en el DFD; por ello, requiere diagramas de comportamiento.

3.6.1. Máquina de estados UML

La máquina de estados UML representa los estados posibles de un objeto, los eventos que provocan cambios, las condiciones que regulan las transiciones y las acciones ejecutadas durante dichos cambios [12, 10]. Este tipo de diagrama es especialmente útil cuando una entidad presenta un ciclo de vida complejo o cuando su estado condiciona qué operaciones están permitidas.

En AquaGest, la entidad más adecuada para una máquina de estados es *Estanque*, porque su estado determina si se puede registrar una siembra, alimentar el cultivo, registrar cosecha o ejecutar mantenimiento. Un ciclo básico podría incluir los estados *Inactivo*, *En preparación*, *Activo*, *En monitoreo*, *En cosecha* y *En mantenimiento*. Por ejemplo, un estanque solo debería pasar a *Activo* cuando se registra una siembra válida, y solo debería pasar a *En cosecha* cuando el administrador habilita el cierre del ciclo productivo.

Además, puede modelarse un estado compuesto para representar la operación activa del estanque. Dentro del estado *Activo*, pueden existir subestados como *Monitoreando agua*, *Registrando alimentación* y *Evaluando crecimiento*. Esta estructura permite cumplir con la exigencia de incluir un estado compuesto y ayuda a representar que, durante la operación productiva, el estanque mantiene actividades recurrentes y simultáneas.

3.6.2. Diagrama de secuencia UML

El diagrama de secuencia UML representa la interacción temporal entre actores, objetos y componentes mediante líneas de vida y mensajes [12, 5]. Su principal utilidad es

mostrar el orden en que ocurren las comunicaciones dentro de un escenario específico. En Ingeniería de Requisitos, este diagrama permite validar que el flujo descrito en los casos de uso se corresponde con las responsabilidades asignadas a cada componente.

En AquaGest, un escenario principal adecuado para el diagrama de secuencia es *Registrar parámetros del agua*. Las líneas de vida pueden incluir *Técnico Acuícola* o *Sensor IoT*, *Interfaz de Monitoreo*, *Servicio de Parámetros*, *Base de Datos* y *Servicio de Alertas*. El flujo inicia con el envío de una medición; luego, el sistema valida el formato, almacena el parámetro, compara el valor contra el rango óptimo y, si el valor está fuera de rango, genera una alerta.

Asimismo, este diagrama puede incluir fragmentos combinados. El fragmento *alt* puede representar dos escenarios: parámetro dentro de rango y parámetro fuera de rango. El fragmento *loop* puede representar la recepción periódica de lecturas desde sensores IoT. De esta manera, el diagrama de secuencia permite comprobar la consistencia entre RF-09, RF-10, RF-11 y el comportamiento esperado del sistema ante mediciones continuas.

3.7. Interrelaciones entre las tres perspectivas

Las perspectivas de datos, funcional y de comportamiento no deben analizarse de forma aislada, porque cada una cubre una dimensión diferente del mismo sistema [1, 5, 11]. La perspectiva de datos identifica qué información debe persistir; la perspectiva funcional describe qué transformaciones se realizan sobre esa información; y la perspectiva de comportamiento explica cómo reaccionan los objetos y componentes ante eventos, condiciones y mensajes.

En AquaGest, esta interrelación puede observarse claramente en el requisito RF-05, relacionado con el registro de siembra. Desde la perspectiva de datos, este requisito exige entidades como *Estanque* y *Siembra*, con una relación uno a muchos. Desde la perspectiva funcional, requiere un proceso que reciba los datos de la siembra, valide capacidad y almacene el registro. Desde la perspectiva de comportamiento, implica que el estanque puede cambiar de estado *En preparación* a *Activo*. Por tanto, un solo requisito puede tener impacto simultáneo en el modelo ER, el DFD, el diagrama de actividades y la máquina de estados.

De igual manera, el requisito RF-10, referido a la alerta por parámetros del agua fuera de rango, también evidencia la complementariedad de modelos. En el modelo de datos, se requieren *ParametroAgua* y *AlertaAgua*. En el modelo funcional, se necesita un proceso de monitoreo que compare valores contra rangos configurados. En el modelo de comportamiento, la alerta debe cambiar de estado desde generada hasta atendida o cerrada. En el diagrama de secuencia, se representa el intercambio de mensajes entre

sensor, servicio de parámetros, base de datos y servicio de alertas.

Tabla 4: Comparación entre las perspectivas de modelado aplicadas a AquaGest

Perspectiva	Notación principal	Aporte al análisis de requisitos	Ejemplo aplicado a AquaGest
Datos	ER Crow's Foot y clases UML	Identifica entidades, atributos, relaciones, cardinalidades y restricciones de integridad.	Relación entre <i>Estanque</i> , <i>Siembra</i> , <i>Grameo</i> , <i>ParametroAgua</i> y <i>Cosecha</i> .
Funcional	DFD y diagrama de actividades UML	Representa procesos, entradas, salidas, almacenes, decisiones y responsabilidades.	Proceso de registrar parámetros del agua, generar alertas y guardar mediciones históricas.
Comportamiento	Máquina de estados y diagrama de secuencia UML	Describe estados, eventos, transiciones, mensajes y escenarios alternativos.	Cambio de estado del estanque desde preparación hasta activo, cosecha, mantenimiento o inactivo.

Finalmente, la integración entre perspectivas permite detectar omisiones y contradicciones. Si una entidad aparece en el modelo de datos pero no está asociada a ningún requisito, podría tratarse de una funcionalidad innecesaria o *gold plating*. Si un requisito no aparece en ningún DFD, clase, actividad o estado, podría existir una brecha de cobertura. Por esta razón, la matriz de trazabilidad $RF \times$ modelos es necesaria para verificar que todos los requisitos relevantes de AquaGest estén representados y que todos los modelos tengan justificación funcional.

4. Revisión y refinamiento del SRS parcial

4.1. Criterios de revisión aplicados al SRS

La revisión del SRS parcial se centró en verificar la calidad de los requisitos funcionales y no funcionales del sistema AquaGest. Para ello, se consideraron criterios como claridad, completitud, consistencia, verificabilidad, trazabilidad, ausencia de ambigüedad y relación directa con las necesidades del dominio camaronero. Estos criterios permiten

detectar si un requisito puede ser comprendido, implementado y validado sin interpretaciones contradictorias.

En primer lugar, se revisó la consistencia interna entre requisitos relacionados. Por ejemplo, se verificó si un requisito que solicita registrar información coincide con otro requisito que calcula dicha información. En segundo lugar, se evaluó la completitud de las descripciones, entradas, salidas, precondiciones, postcondiciones y criterios de verificación. En tercer lugar, se revisó la verificabilidad, es decir, si el requisito contiene condiciones medibles que permitan comprobar su cumplimiento mediante pruebas.

Además, se analizaron posibles ambigüedades en términos como “crecimiento inferior al esperado”, “inducción básica” o “capacidad de siembra”, debido a que estas expresiones pueden generar interpretaciones diferentes entre el equipo de desarrollo, los usuarios y los evaluadores. Por tanto, cada defecto identificado se clasificó según su tipo y se propuso una corrección concreta para mejorar la calidad del SRS.

4.2. Registro de defectos identificados

A partir de la revisión del SRS parcial de AquaGest, se identificaron defectos relacionados con inconsistencia, incompletitud y ambigüedad. La Tabla 5 presenta el registro de defectos con su respectiva corrección propuesta.

Tabla 5: Registro de defectos del SRS con corrección propuesta.

ID	Defecto	Tipo	Corrección propuesta
DF-01	La descripción de RF-01 incluye “capacidad de siembra” como dato a registrar en el estanque, pero RF-02 establece que la capacidad se calcula y no se ingresa manualmente.	Inconsistencia	Eliminar “capacidad de siembra” de las entradas de RF-01 y dejar el cálculo exclusivamente en RF-02.
DF-02	El criterio de verificación de RF-02 aparece incompleto: “el sistema sugiere 1 500 000 larvas (± 5 ”, sin indicar la unidad o porcentaje de tolerancia.	Incompletitud	Completar el criterio como “ $\pm 5\%$ de tolerancia”.

ID	Defecto	Tipo	Corrección propuesta
DF-03	El rango de sobrecarga controlada, definido como 1000–2000 lb, solo aparece en el criterio de verificación de RF-03 y no en la descripción del requisito.	Ambigüedad / Incompletitud	Incorporar el rango cuantificado dentro de la descripción del requisito.
DF-04	El requisito RF-08 utiliza la expresión “crecimiento inferior al esperado”, pero no define el umbral dentro de la descripción. El valor de 2 g solo aparece en el criterio de verificación.	Ambigüedad	Cuantificar el umbral directamente en la descripción del requisito, indicando que la alerta se genera si la ganancia es menor a 2 g.
DF-05	El criterio de verificación de RF-18 exige una regla de control de acceso, pero dicha regla no aparece en la descripción ni en las entradas del requisito.	Incompletitud	Agregar explícitamente la regla de autorización: solo el empleado asignado o su supervisor directo pueden marcar una tarea como completada.
DF-06	El RNF-03 emplea el término “inducción básica”, pero no especifica duración ni contenido mínimo, lo que afecta su verificabilidad.	Ambigüedad	Definir la inducción como una capacitación máxima de 30 minutos sobre el módulo de alimentación o eliminar la referencia no medible.

4.3. Requisitos corregidos con sintaxis EARS

Luego de identificar los defectos del SRS parcial, se reformularon los requisitos afectados utilizando una redacción más precisa y verificable. La Tabla 6 presenta la versión corregida de los requisitos seleccionados.

Tabla 6: Requisitos refinados mediante sintaxis EARS.

ID	Redacción original	Redacción corregida
RF-01	El sistema debe permitir registrar cada piscina con número/identificación único, tamaño en hectáreas, capacidad de siembra y estado actual.	El sistema deberá permitir registrar cada estanque con ID único, tamaño en hectáreas y estado inicial. La capacidad de siembra deberá calcularse a partir del tamaño registrado, conforme al requisito RF-02.
RF-02	El sistema debe calcular la capacidad máxima de larvas por estanque según su superficie.	El sistema deberá calcular la capacidad máxima de larvas por estanque en función de su superficie, con una tolerancia de $\pm 5\%$ respecto al valor de referencia de 1 500 000 larvas por 5 ha.
RF-03	Permite registrar sobrecarga del estanque con justificación.	Mientras exista una siembra activa en el estanque, el sistema deberá permitir registrar una sobrecarga controlada de entre 1000 y 2000 libras, exigiendo una justificación obligatoria para valores dentro de ese rango.
RF-08	Generar alerta si el crecimiento es inferior al esperado.	Mientras existan al menos dos grameos consecutivos registrados, el sistema deberá generar una alerta si la ganancia de peso entre grameos es menor a 2 g, en un plazo no mayor a 24 horas.
RF-18	El sistema debe permitir asignar tareas diarias a empleados por área y registrar su cumplimiento.	El sistema deberá permitir asignar tareas diarias a empleados por área y registrar su estado de cumplimiento. El sistema deberá rechazar cualquier intento de marcar una tarea como completada por un usuario distinto al empleado asignado o a su supervisor directo.

ID	Redacción original	Redacción corregida
RNF-03	Un operario de producción debe poder completar el registro de alimentación diaria en un máximo de 3 pasos y 60 segundos, sin capacitación previa más allá de la inducción básica.	El sistema deberá permitir que un operario de producción complete el registro de alimentación diaria en un máximo de 3 pasos y 60 segundos, sin haber recibido capacitación previa distinta de una inducción máxima de 30 minutos sobre el módulo.

4.4. Tabla de atributos completa de RF y RNF

La tabla de atributos permite controlar y verificar cada requisito del sistema. Para esta práctica se consideran los atributos solicitados: ID, descripción, tipo, prioridad MoSCoW, fuente, estabilidad, estado y verificabilidad. La Tabla 7 presenta los requisitos funcionales y la Tabla 8 presenta los requisitos no funcionales.

Tabla 7: Tabla de atributos de requisitos funcionales.

ID	Descripción	Tipo	MoSCoW	Fuente	Estabilidad	Estado	Verif.
RF-01	Registro de estanque con ID, tamaño y estado.	RF	Must	EV-02	Alta	Validado	S
RF-02	Cálculo de capacidad máxima de siembra.	RF	Must	EV-01	Alta	Validado	S
RF-03	Registro de sobrecarga controlada de 1000–2000 lb.	RF	Should	EV-01	Media	Validado	S
RF-04	Actualización de estado del estanque.	RF	Must	EV-02	Alta	Validado	S
RF-05	Registro de siembra.	RF	Must	EV-02	Alta	Validado	S
RF-06	Registro de grameo semanal.	RF	Must	EV-01	Alta	Validado	S
RF-07	Historial de crecimiento.	RF	Should	EV-01	Media	Validado	S
RF-08	Alerta de crecimiento inferior al esperado, menor a 2 g.	RF	Should	EV-01	Media	Validado	S
RF-09	Registro de parámetros del agua.	RF	Must	EV-02	Alta	Validado	S
RF-10	Alerta por parámetro fuera de rango.	RF	Must	EV-01, EV-02	Alta	Validado	S
RF-11	Registro de acción correctiva.	RF	Should	EV-02	Media	Validado	S
RF-12	Cálculo de ración diaria.	RF	Must	EV-01, EV-02	Alta	Validado	S
RF-13	Registro de consumo de alimento.	RF	Must	EV-02	Alta	Validado	S
RF-14	Informe de eficiencia alimenticia FCR.	RF	Should	EV-01	Media	Validado	S
RF-15	Registro de incidente sanitario.	RF	Must	EV-01	Alta	Validado	S
RF-16	Alerta de mortalidad anormal.	RF	Must	EV-01, EV-02	Alta	Validado	S
RF-17	Registro de empleados.	RF	Must	EV-02	Alta	Validado	S
RF-18	Asignación y control de tareas con regla de autorización.	RF	Should	EV-02	Media	Validado	S
RF-19	Gestión de inventario de insumos.	RF	Must	EV-02	Alta	Validado	S
RF-20	Alerta de stock mínimo.	RF	Should	EV-02	Media	Validado	S
RF-21	Registro de órdenes de mantenimiento.	RF	Could	EV-02	Baja	Propuesto	S
RF-22	Registro de cosecha.	RF	Must	EV-02	Alta	Validado	S
RF-23	Cálculo de costo de producción.	RF	Should	EV-02	Media	Validado	S
RF-24	Reportes comparativos de producción.	RF	Should	EV-01, EV-02	Media	Validado	S
RF-25	Predicción de momento óptimo de cosecha.	RF	Could	EV-01	Baja	Propuesto	S

Tabla 8: Tabla de atributos de requisitos no funcionales.

ID	Descripción	Tipo	MoSCoW	Fuente	Estabilidad	Estado	Verif.
RNF-01	Eficiencia de desempeño: registro de parámetros en tiempo menor o igual a 2 s con 50 usuarios concurrentes.	RNF	Must	EV-02	Alta	Validado	S

ID	Descripción	Tipo	MoSCoW	Fuente	Estabilidad	Estado	Verif.
RNF-02	Disponibilidad mensual mayor o igual al 98 %.	RNF	Must	EV-01	Alta	Validado	S
RNF-03	Usabilidad: registro de alimentación en máximo 3 pasos y 60 segundos.	RNF	Should	EV-02	Media	Validado	S
RNF-04	Seguridad: control RBAC y cifrado de contraseñas mediante bcrypt o equivalente.	RNF	Must	Sección 2.6	Alta	Validado	S
RNF-05	Tolerancia a fallos mediante captura offline y sincronización posterior.	RNF	Should	Sección 2.5	Media	Validado	S
RNF-06	Compatibilidad con Chrome, Firefox y tablet Android.	RNF	Could	Sección 2.5	Baja	Propuesto	S
RNF-07	Mantenibilidad mediante cobertura de pruebas unitarias mayor o igual al 60 % en módulos críticos.	RNF	Could	Decisión de equipo	Baja	Propuesto	S

5. Modelo de datos: ER y clases UML

5.1. Identificación de entidades del dominio

Las entidades del dominio fueron extraídas de los requisitos funcionales de AquaGest, especialmente de aquellos relacionados con la gestión de estanques, siembras, grameos, parámetros del agua, alimentación, inventario, empleados y cosechas. La Tabla 9 presenta las entidades identificadas, sus atributos principales, tipos de datos, restricciones y clave primaria.

Tabla 9: Entidades del modelo de datos.

Entidad	Atributos	Tipos y restricciones	Clave primaria
<i>Estanque</i>	id_estanque, nombre, tamaño_ha, estado.	id_estanque: int autoincremental; nombre: varchar(50), NOT NULL; tamaño_ha: decimal(6,2), mayor que 0; estado: enum(activo, preparacion, cosecha, inactivo).	id_estanque
<i>Siembra</i>	id_siembra, fecha_inicio, cantidad_larvas, tipo_larva, laboratorio.	id_siembra: int autoincremental; fecha_inicio: date, NOT NULL; cantidad_larvas: int, mayor que 0; tipo_larva: varchar(30); laboratorio: varchar(50).	id_siembra
<i>Grameo</i>	id_grameo, fecha, peso_promedio_g, observaciones.	id_grameo: int autoincremental; fecha: date, NOT NULL; peso_promedio_g: decimal(5,2), mayor que 0; observaciones: text, nullable.	id_grameo

Entidad	Atributos	Tipos y restricciones	Clave primaria
<i>ParametroAgua</i>	id_parametro, fecha_hora, tipo_parametro, valor.	id_parametro: int autoincremental; fecha_hora: datetime, NOT NULL; tipo_parametro: enum(pH, oxigeno, temperatura, salinidad, turbidez, alcalinidad); valor: decimal(6,2).	id_parametro
<i>IncidenteSanitario</i>	id_incidente, fecha, sintomas, diagnostico, tratamiento.	id_incidente: int autoincremental; fecha: date; sintomas: text; diagnostico: varchar(100); tratamiento: text, nullable.	id_incidente
<i>Empleado</i>	id_empleado, cedula, nombre, cargo, rol_sistema.	id_empleado: int autoincremental; cedula: char(10), UNIQUE, NOT NULL; nombre: varchar(80); cargo: varchar(50); rol_sistema: enum RBAC.	id_empleado
<i>Insumo</i>	id_insumo, nombre, categoria, stock_actual, stock_minimo.	id_insumo: int autoincremental; nombre: varchar(50); categoria: enum(balanceado, medicina, herramienta, combustible); stock_actual: decimal(8,2), mayor o igual que 0; stock_minimo: decimal(8,2).	id_insumo
<i>MovimientoInventario</i>	id_movimiento, fecha, tipo_movimiento, cantidad.	id_movimiento: int autoincremental; fecha: date; tipo_movimiento: enum(entrada, salida); cantidad: decimal(8,2), mayor que 0.	id_movimiento

Entidad	Atributos	Tipos y restricciones	Clave primaria
<i>Cosecha</i>	id_cosecha, fecha, peso_total_kg, rendimiento.	id_cosecha: int autoincremental; fecha: date; peso_total_kg: decimal(8,2), mayor que 0; rendimiento: decimal(6,2).	id_cosecha
<i>RegistroAlimentacion</i>	id_registro, fecha, cantidad_kg.	id_registro: int autoincremental; fecha: date; cantidad_kg: decimal(6,2), mayor que 0.	id_registro
<i>AsignacionEstanque</i>	id_asignacion, fecha_inicio, fecha_fin, rol_asignado.	id_asignacion: int autoincremental; fecha_inicio: date; fecha_fin: date, nullable; rol_asignado: varchar(30).	id_asignacion

5.2. Relaciones, cardinalidades y participación

Las relaciones del modelo ER se definieron considerando la interacción lógica entre las entidades principales del sistema AquaGest. Se incorporaron relaciones uno a muchos, relaciones opcionales y dos relaciones muchos a muchos resueltas mediante entidades asociativas. Esto permite cumplir con el criterio de modelado solicitado para la PE3 y, a la vez, conservar coherencia con los requisitos funcionales del SRS parcial.

Tabla 10: Relaciones del modelo ER.

Relación	Cardinalidad	Participación / regla
<i>Estanque registra Siembra</i>	1 : N	Un estanque puede registrar cero o muchas siembras durante su vida operativa; cada siembra pertenece obligatoriamente a un estanque.
<i>Siembra registra Grameo</i>	1 : N	Una siembra debe permitir varios grameos durante el ciclo productivo; cada grameo pertenece obligatoriamente a una siembra.
<i>Estanque mide ParametroAgua</i>	1 : N	Un estanque puede tener muchas mediciones de parámetros de agua; cada medición pertenece obligatoriamente a un estanque.

Relación	Cardinalidad	Participación / regla
<i>Siembra presenta IncidenteSanitario</i>	1 : N	Una siembra puede presentar cero o varios incidentes sanitarios; cada incidente sanitario debe asociarse a una siembra específica.
<i>Estanque produce Cosecha</i>	1 : N	Un estanque puede producir varias cosechas a lo largo del tiempo; cada cosecha debe estar asociada a un estanque.
<i>Siembra culmina en Cosecha</i>	1 : 0..1	Una siembra puede culminar en una cosecha, aunque no necesariamente todas las siembras tienen una cosecha registrada al momento del análisis.
<i>Insumo mueve MovimientoInventario</i>	1 : N	Un insumo puede tener múltiples movimientos de entrada o salida; cada movimiento corresponde obligatoriamente a un insumo.
<i>Empleado ejecuta MovimientoInventario</i>	1 : N	Un empleado puede registrar varios movimientos de inventario; cada movimiento debe quedar asociado al empleado responsable.
<i>Estanque consume Insumo, resuelta por RegistroAlimentacion</i>	N : M	Un estanque puede consumir varios tipos de insumo, especialmente balanceado, y un mismo insumo puede utilizarse en varios estanques. La relación se resuelve mediante la entidad asociativa <i>RegistroAlimentacion</i> .
<i>Empleado se asigna a Estanque, resuelta por AsignacionEstanque</i>	N : M	Un empleado puede ser asignado a varios estanques en distintos turnos o roles, y un estanque puede tener varios empleados asignados. La relación se resuelve mediante la entidad asociativa <i>AsignacionEstanque</i> .

Las dos entidades asociativas, *RegistroAlimentacion* y *AsignacionEstanque*, poseen atributos propios además de las claves foráneas correspondientes. Por esta razón, no se consideran simples tablas puente, sino entidades asociativas con significado dentro del dominio de AquaGest. La primera permite registrar fecha y cantidad de alimento suministrado, mientras que la segunda permite controlar fecha de inicio, fecha de fin y rol asignado del empleado en el estanque.

5.3. Verificación de tercera forma normal

La verificación de tercera forma normal se aplicó para comprobar que el modelo de datos reduzca redundancias, evite dependencias innecesarias y conserve la integridad de la información. En el caso de AquaGest, la revisión se realizó sobre las once entidades identificadas, considerando la atomicidad de los atributos, la dependencia respecto de la clave primaria y la ausencia de dependencias transitivas entre atributos no clave.

Tabla 11: Verificación de normalización del modelo de datos.

Entidad / criterio	Forma normal revisada	Observación
Todas las entidades	Primera forma normal	Los atributos son atómicos, no se registran listas dentro de un mismo campo y cada entidad posee una clave primaria definida.
Todas las entidades	Segunda forma normal	No existen claves primarias compuestas que generen dependencias parciales, debido a que las entidades utilizan identificadores simples autoincrementales.
<i>Estanque</i>	Tercera forma normal	La capacidad máxima de siembra no se almacena como columna fija, debido a que se calcula a partir de <i>tamano_ha</i> , conforme al RF-02. Esto evita redundancia derivada.
<i>Cosecha</i>	Tercera forma normal	El rendimiento se documenta como dato calculado o cacheado. La fuente de verdad se mantiene en los valores de peso total y superficie del estanque.
<i>RegistroAlimentacion</i>	Tercera forma normal	No repite datos del insumo, como categoría o nombre, ni datos del estanque, como tamaño o estado. Solo referencia las claves correspondientes.
<i>MovimientoInventario</i>	Tercera forma normal	El movimiento almacena únicamente fecha, tipo y cantidad. Los datos descriptivos del insumo permanecen en la entidad <i>Insumo</i> .
Resultado general	3FN cumplida	El modelo queda verificado en tercera forma normal, debido a que no se identifican dependencias transitivas no justificadas entre atributos no clave.

En consecuencia, el modelo de datos de AquaGest cumple con 1FN, 2FN y 3FN. Esta

verificación permite sostener que el diseño evita redundancias innecesarias y facilita la trazabilidad de la información entre siembras, grameos, parámetros del agua, alimentación, inventario y cosechas.

5.4. Diagrama ER en notación Crow's Foot

El Diagrama ER en notación Crow's Foot representa gráficamente las entidades, atributos, claves primarias, relaciones, cardinalidades y participaciones del modelo de datos. Para AquaGest, este diagrama debe reflejar las once entidades identificadas, las relaciones uno a muchos y las dos relaciones muchos a muchos resueltas mediante *RegistroAlimentacion* y *AsignacionEstanque*.

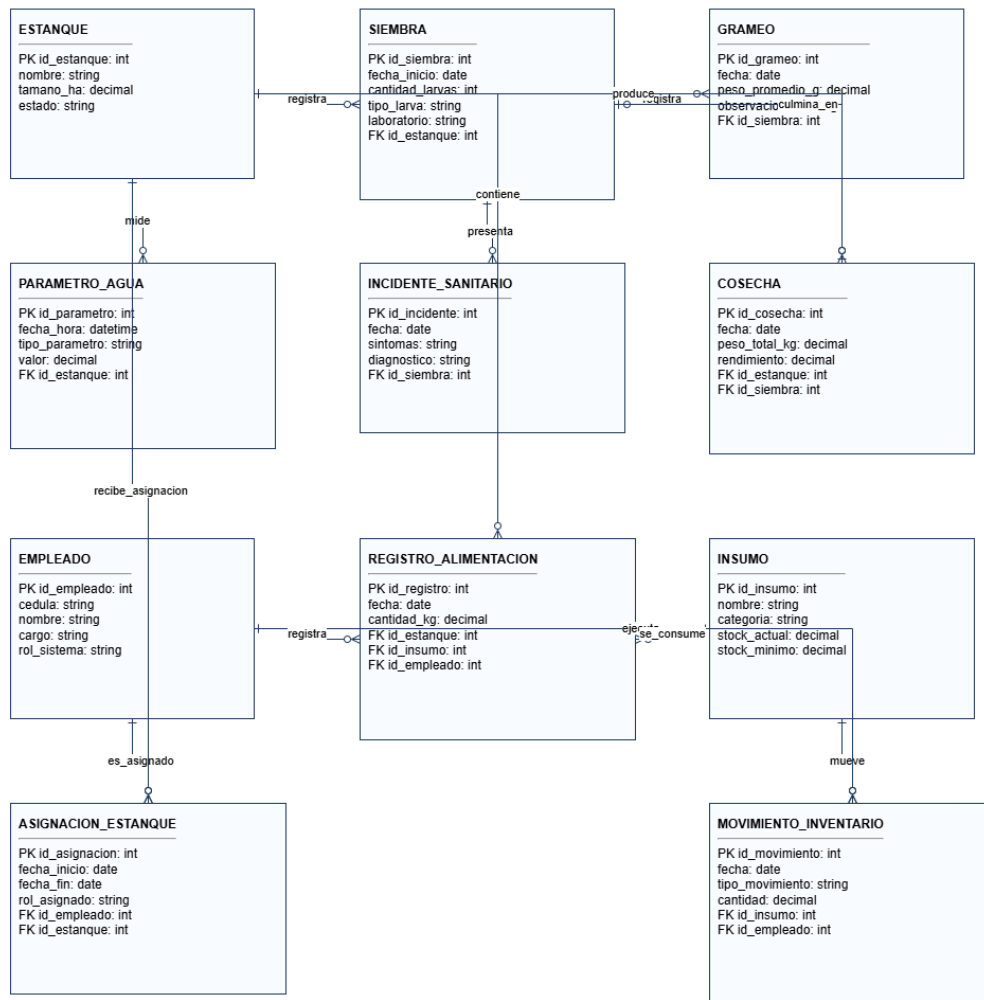


Figura 1: Diagrama ER del sistema AquaGest en notación Crow's Foot.

Para la construcción del diagrama se deben considerar las siguientes reglas: cada entidad debe incluir su clave primaria, atributos principales y restricciones relevantes; las relaciones deben mostrar cardinalidad y opcionalidad; y las entidades asociativas deben visualizarse como elementos que resuelven las relaciones N:M. Esto permite que el diagrama ER sea coherente con la tabla de entidades y con los requisitos funcionales asociados al modelo de datos.

5.5. Transformación del ER al Diagrama de Clases UML

La transformación del modelo ER al Diagrama de Clases UML se realizó conservando las entidades principales como clases del dominio. Los atributos del modelo ER se trasladan como atributos privados de clase, las claves primarias se mantienen como identificadores y las relaciones se convierten en asociaciones UML con multiplicidades. Además, las entidades asociativas se representan como clases con estereotipo «Asociativa», debido a que contienen atributos propios y resuelven relaciones N:M.

Tabla 12: Transformación de entidades ER a clases UML.

Entidad ER	Clase UML	Criterio de transformación
<i>Estanque</i>	Estanque	Se transforma en clase principal del dominio. Sus atributos se mantienen privados. Operaciones sugeridas: <code>calcularCapacidadMaxima()</code> y <code>actualizarEstado()</code> .
<i>Siembra</i>	Siembra	Representa el ciclo de cultivo iniciado en un estanque. Operaciones sugeridas: <code>registrarSiembra()</code> y <code>cerrarSiembra()</code> .
<i>Grameo</i>	Grameo	Permite registrar el peso promedio del camarón durante el ciclo productivo. Operaciones sugeridas: <code>registrarPeso()</code> y <code>calcularGanancia()</code> .
<i>ParametroAgua</i>	ParametroAgua	Representa las mediciones de calidad del agua. Operaciones sugeridas: <code>registrarParametro()</code> y <code>validarRango()</code> .
<i>IncidenteSanitario</i>	IncidenteSanitario	Representa eventos sanitarios asociados a una siembra. Operaciones sugeridas: <code>registrarIncidente()</code> y <code>actualizarTratamiento()</code> .

Entidad ER	Clase UML	Criterio de transformación
<i>Empleado</i>	Empleado	Representa al personal vinculado al sistema. Operaciones sugeridas: asignarRol() y validarPermiso() .
<i>Insumo</i>	Insumo	Representa los recursos usados en alimentación, sanidad, mantenimiento o producción. Operaciones sugeridas: actualizarStock() y estaBajoMinimo() .
<i>MovimientoInventario</i>	MovimientoInventario	Representa entradas y salidas de insumos. Operaciones sugeridas: registrarEntrada() y registrarSalida() .
<i>Cosecha</i>	Cosecha	Representa el cierre productivo de un ciclo. Operaciones sugeridas: registrarCosecha() y calcularRendimiento() .
<i>RegistroAlimentacion</i>	RegistroAlimentacion «Asociativa»	Clase asociativa que resuelve la relación N:M entre <i>Estanque</i> e <i>Insumo</i> . Operaciones sugeridas: registrarConsumo() y calcularRacion() .
<i>AsignacionEstanque</i>	AsignacionEstanque «Asociativa»	Clase asociativa que resuelve la relación N:M entre <i>Empleado</i> y <i>Estanque</i> . Operaciones sugeridas: asignarEmpleado() y cerrarAsignacion() .

Durante la transformación, se aplican visibilidades UML colocando los atributos como privados mediante el símbolo “-”. Las operaciones se declaran como públicas mediante el símbolo “+”. Las multiplicidades del modelo ER se trasladan sin cambios hacia las asociaciones UML, por ejemplo, *Estanque* 1..* *Siembra*, *Siembra* 1..* *Grameo* y *Estanque* 1..* *ParametroAgua*. Además, las clases **RegistroAlimentacion** y **AsignacionEstanque** se mantienen como clases asociativas debido a que contienen datos propios y no son únicamente enlaces técnicos.

5.6. Diagrama de Clases UML

El Diagrama de Clases UML representa la transformación del modelo ER hacia una estructura orientada a objetos. Para AquaGest, el diagrama debe incluir las clases derivadas de las entidades del modelo de datos, sus atributos con tipos, operaciones principales, visibilidades y multiplicidades. También debe representar explícitamente las clases asociativas **RegistroAlimentacion** y **AsignacionEstanque**, debido a que resuelven las relaciones muchos a muchos exigidas para el modelo.

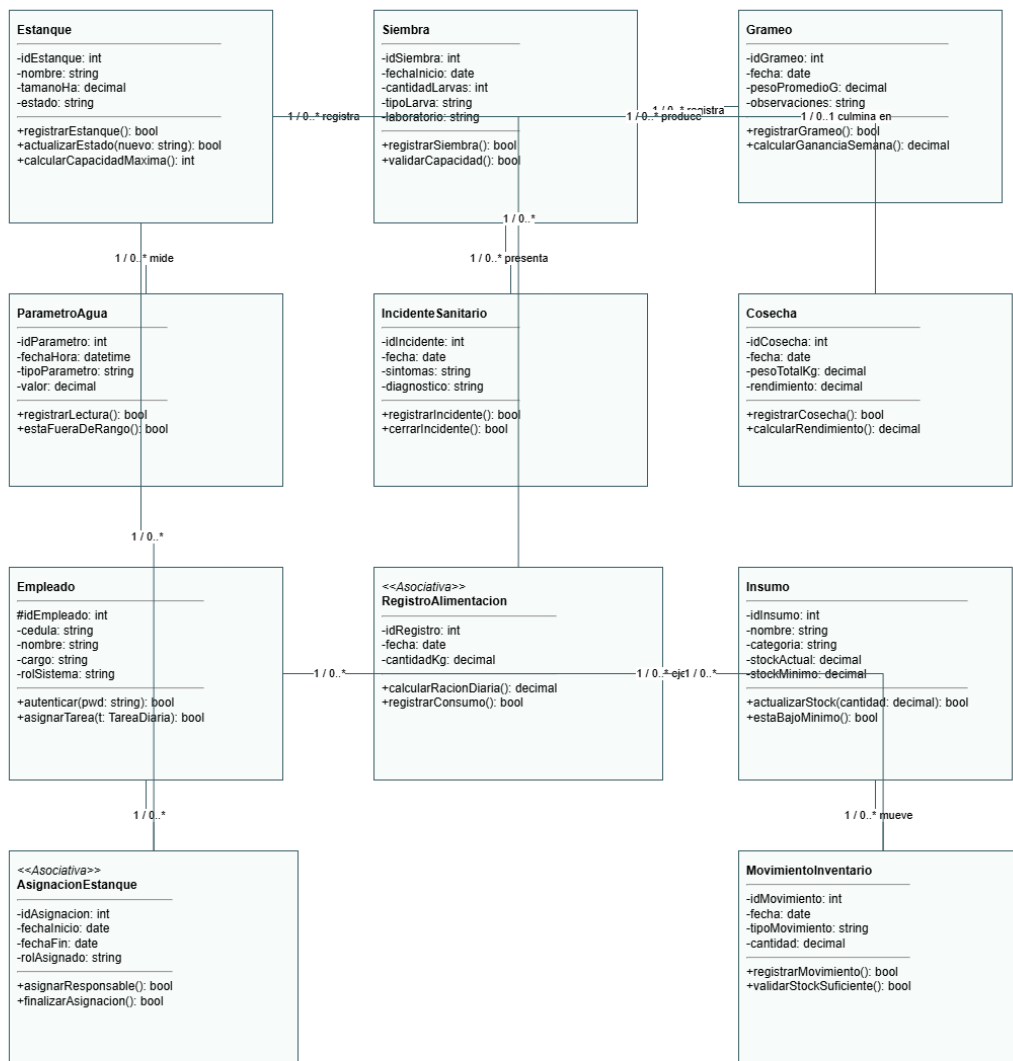


Figura 2: Diagrama de Clases UML del sistema AquaGest.

Para cumplir con la rúbrica de evaluación, cada clase del diagrama debe contener al menos dos operaciones, sus atributos deben incluir tipo de dato y visibilidad, y las relaciones deben reflejar las multiplicidades ya definidas en el modelo ER. De esta manera, el Diagrama de Clases UML mantiene coherencia con la perspectiva de datos y permite preparar la transición hacia los modelos funcionales y de comportamiento.

6. Modelo funcional: DFD y actividades UML

6.1. DFD Nivel 0: diagrama de contexto

El DFD Nivel 0, también denominado diagrama de contexto, representa a AquaGest como un único proceso central que interactúa con entidades externas. Este nivel permite delimitar el alcance del sistema y diferenciar los actores externos que envían información, reciben resultados o intercambian datos con la plataforma.

Para AquaGest, el proceso central se denomina *0 – Sistema AquaGest*. Las entidades externas identificadas son: Administrador, Técnico Acuícola, Sensores IoT, Módulo de IA, Operario de Producción, Operador de Maquinaria, Supervisor Técnico de Mantenimiento y Bodeguero. Cada entidad se conecta con el sistema mediante flujos de datos coherentes con los requisitos funcionales previamente definidos.

Tabla 13: Flujos de datos del DFD Nivel 0.

Entidad externa	Flujo entrante a AquaGest	Flujo saliente de AquaGest
Administrador	Configuración de estanques, empleados, roles y parámetros generales.	Reportes operativos, reportes comparativos y alertas del sistema.
Técnico Acuícola	Registro de parámetros de agua, grameos, incidentes sanitarios y acciones correctivas.	Alertas de agua, alertas sanitarias e historial de monitoreo.
Sensores IoT	Lecturas de O ₂ , pH, temperatura, salinidad, turbidez y alcalinidad.	No aplica, debido a que los sensores actúan como fuente de datos de entrada.
Módulo de IA	Datos históricos, variables actuales del cultivo y registros productivos.	Predicciones, alertas tempranas y recomendaciones de cosecha.
Operario de Producción	Registro de siembra, alimentación diaria y consumo de insumos.	Raciones sugeridas, confirmaciones de registro y alertas operativas.
Operador de Maquinaria	Reporte de fallas de equipos o novedades de maquinaria.	Orden de trabajo o estado de atención de la novedad.
Supervisor Técnico de Mantenimiento	Registro y seguimiento de órdenes de mantenimiento.	Estado de órdenes, historial de mantenimiento y notificaciones.

Entidad externa	Flujo entrante a AquaGest	Flujo saliente de AquaGest
Bodeguero	Movimientos de inventario, entradas y salidas de insumos.	Alertas de stock mínimo y estado actualizado del inventario.

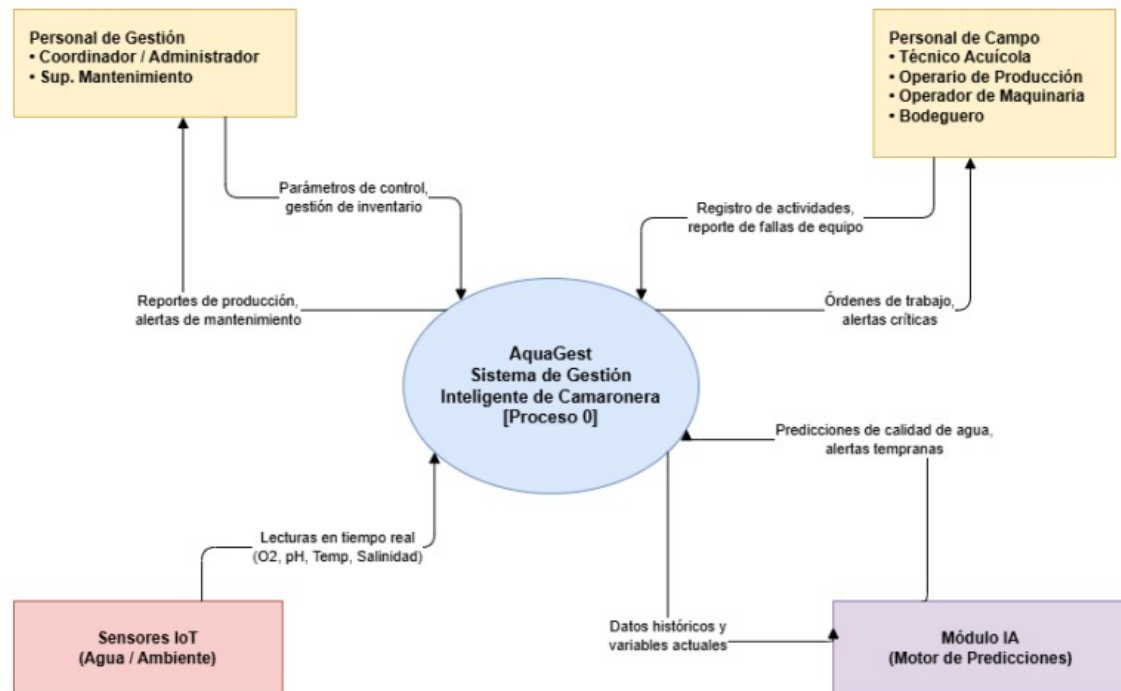


Figura 3: DFD Nivel 0: diagrama de contexto del sistema AquaGest.

6.2. DFD Nivel 1

El DFD Nivel 1 descompone el proceso central *Sistema AquaGest* en subprocesos internos. Para esta práctica se definieron ocho subprocesos, superando el mínimo solicitado de cinco subprocesos. Estos procesos se agrupan según los módulos funcionales del sistema y se relacionan con los requisitos funcionales RF-01 a RF-25.

Asimismo, el DFD Nivel 1 incorpora almacenes internos de datos que permiten representar la persistencia de información dentro del sistema. Estos almacenes se relacionan con estanques, siembras, grameos, parámetros de agua, alimentación, sanidad, empleados, inventario, mantenimiento, cosechas y predicción mediante IA.

Tabla 14: Subprocesos y almacenes internos del DFD Nivel 1.

Subproceso	RF que cubre	Almacenes que utiliza
1.0 Gestionar Estanques y Siembras	RF-01 a RF-08	D1 Estanques; D2 Siembras y Grameos.
2.0 Monitorear Parámetros del Agua	RF-09 a RF-11	D3 Parámetros de Agua; D10 Alertas de Agua.
3.0 Gestionar Alimentación	RF-12 a RF-14	D4 Alimentación; consulta D2 Siembras y Grameos.
4.0 Gestionar Sanidad	RF-15 y RF-16	D5 Historial Sanitario; consulta D2 Siembras y Grameos.
5.0 Gestionar Empleados y Tareas	RF-17 y RF-18	D6 Empleados y Tareas.
6.0 Gestionar Inventario y Mantenimiento	RF-19 a RF-21	D7 Inventario; D8 Mantenimiento.
7.0 Gestionar Cosechas y Reportes	RF-22 a RF-24	D9 Cosechas; consulta D1, D2, D4 y D7.
8.0 Predicción IA de Cosecha	RF-25	Consulta D2 Siembras y Grameos, D3 Parámetros de Agua y D9 Cosechas.

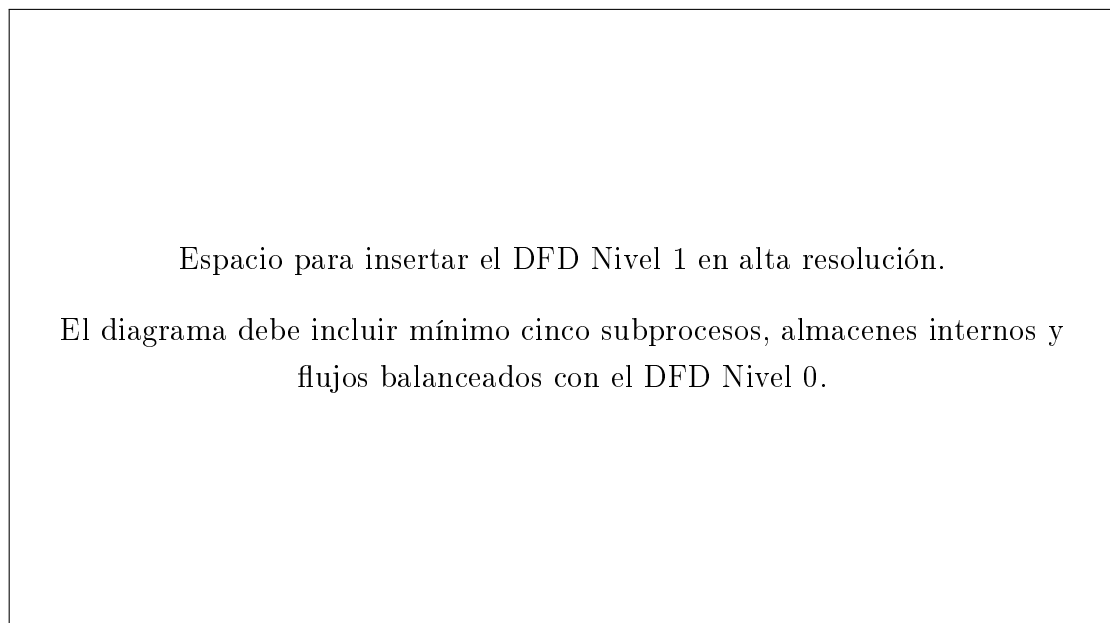


Figura 4: DFD Nivel 1 del sistema AquaGest.

6.3. DFD esencial del proceso principal

El DFD esencial representa la lógica del negocio sin enfocarse en detalles tecnológicos, interfaces, sensores específicos o dispositivos de captura. Para AquaGest, el proceso esencial seleccionado corresponde al ciclo productivo completo del cultivo, debido a que integra la secuencia principal de operación: siembra, monitoreo de agua, alimentación, sanidad y cosecha.

Este proceso se considera crítico porque sostiene los requisitos funcionales principales del sistema. Sin este flujo, AquaGest no lograría reemplazar de forma adecuada el registro manual actual ni aportar trazabilidad al proceso productivo de la camaronera.

Tabla 15: DFD esencial del proceso de negocio principal.

Evento o estímulo	Actividad esencial	Memoria que usa	Respuesta al entorno
Se decide iniciar una siembra.	E-A1 Iniciar ciclo productivo.	Crea M1; consulta M2.	Confirmación de siembra registrada.
Llega una lectura de parámetro de agua.	E-A2 Vigilar calidad del agua.	Lee y escribe M1.	Alerta de agua fuera de rango, cuando corresponda.
Se cumple el intervalo de grameo semanal.	E-A3 Evaluar crecimiento.	Lee y escribe M1.	Alerta de crecimiento insuficiente.
Llega el momento de alimentar.	E-A4 Suministrar alimentación.	Lee y escribe M1.	Ración diaria calculada y registrada.
Se observa un síntoma sanitario.	E-A5 Atender incidente sanitario.	Lee y escribe M1.	Alerta de mortalidad o tratamiento registrado.
El cultivo alcanza talla o peso objetivo.	E-A6 Ejecutar cosecha.	Lee M1; escribe M2.	Reporte de cosecha y rendimiento.

Las memorias esenciales utilizadas en este modelo son las siguientes:

- **M1 – Memoria del cultivo:** contiene el estado del cultivo, siembra, grameos, parámetros de agua, alimentación y sanidad. Se considera una sola memoria lógica porque todas estas actividades comparten el mismo dato de negocio: el cultivo vivo en curso.
- **M2 – Memoria de cosechas:** contiene el histórico de rendimiento por ciclo. Es consultada al iniciar un nuevo ciclo productivo y se actualiza cuando se ejecuta la cosecha.

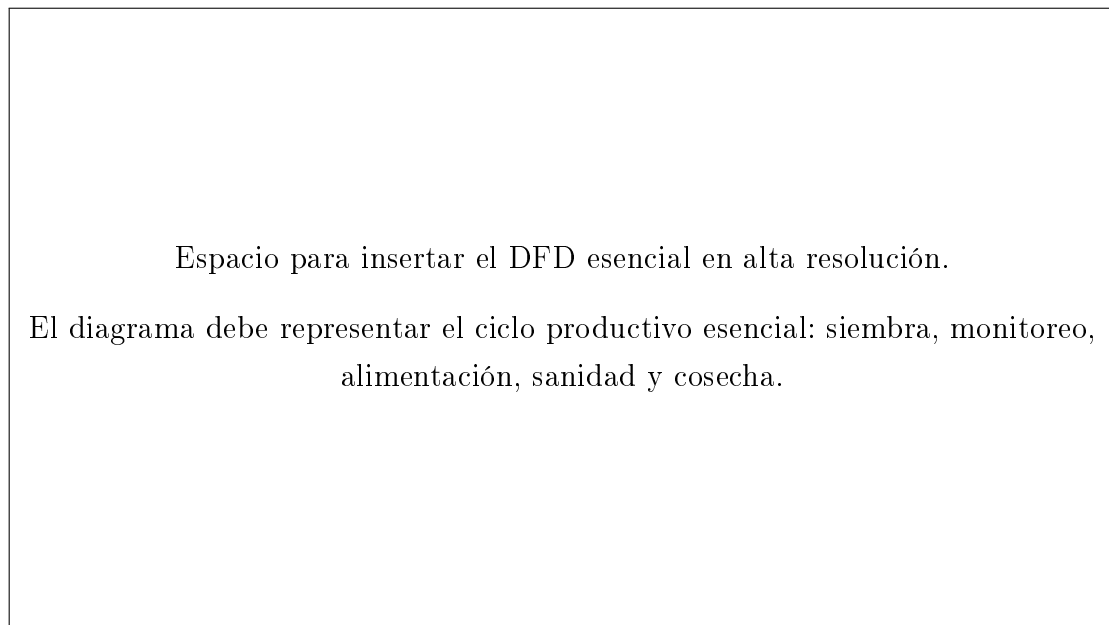


Figura 5: DFD esencial del proceso principal del sistema AquaGest.

6.4. Balanceo entre DFD Nivel 0 y DFD Nivel 1

El balanceo entre el DFD Nivel 0 y el DFD Nivel 1 permite verificar que los flujos externos identificados en el diagrama de contexto se conserven en la descomposición interna del sistema. En este sentido, el Nivel 1 no debe crear ni eliminar flujos externos sin justificación, sino redistribuirlos entre los subprocesos correspondientes.

En AquaGest, cada entidad externa mantiene los mismos flujos netos de entrada y salida definidos en el DFD Nivel 0. El DFD Nivel 1 únicamente reparte dichos flujos entre los subprocesos de gestión de estanques, monitoreo de agua, alimentación, sanidad, empleados, inventario, mantenimiento, cosechas, reportes y predicción mediante IA.

Tabla 16: Verificación de balanceo entre DFD Nivel 0 y DFD Nivel 1.

Entidad externa	Flujo en Nivel 0	Subproceso equivalente en Nivel 1
Administrador	Configuración de estanques y empleados ↔ reportes y alertas.	1.0 y 5.0 reciben configuración; 7.0 genera reportes; 2.0, 4.0 y 6.0 generan alertas.
Técnico Acuícola	Parámetros de agua y grameo ↔ alertas de agua y sanidad.	1.0 recibe grameos; 2.0 recibe parámetros y genera alertas de agua; 4.0 gestiona alertas sanitarias.

Entidad externa	Flujo en Nivel 0	Subproceso equivalente en Nivel 1
Sensores IoT	Lecturas de O ₂ , pH, temperatura, salinidad, turbidez y alcalinidad → sistema.	2.0 Monitorear Parámetros del Agua recibe las lecturas. No existe flujo de retorno, igual que en el Nivel 0.
Módulo de IA	Datos históricos y variables actuales ↔ predicciones y alertas tempranas.	8.0 Predicción IA de Cosecha recibe datos históricos y devuelve predicciones.
Operario de Producción	Registro de siembra y alimentación ↔ raciones sugeridas.	1.0 recibe registros de siembra; 3.0 recibe alimentación y devuelve ración sugerida.
Operador de Maquinaria	Reporte de fallas de equipo ↔ orden de trabajo.	6.0 Gestionar Inventario y Mantenimiento recibe fallas y genera orden de trabajo.
Supervisor Técnico de Mantenimiento	Registro de órdenes de mantenimiento ↔ estado de órdenes.	6.0 recibe órdenes de mantenimiento y devuelve estado actualizado.
Bodeguero	Movimientos de inventario ↔ alertas de stock mínimo.	6.0 recibe movimientos de inventario y genera alertas de stock mínimo.

Con base en esta revisión, se determina que el DFD Nivel 1 se encuentra balanceado con el DFD Nivel 0, debido a que conserva las entidades externas y sus flujos principales. Las diferencias observadas corresponden únicamente a la descomposición interna de procesos, lo cual es válido dentro del refinamiento funcional del modelo.

6.5. Diagrama de Actividades UML del caso de uso principal

El diagrama de actividades UML se desarrolla sobre el caso de uso principal *Registrar Parámetros del Agua*, debido a que este proceso se relaciona directamente con el monitoreo productivo, la generación de alertas y la toma de decisiones técnicas dentro de la camaronera. El flujo involucra tres responsabilidades principales: la captura del dato, la validación del sistema y la respuesta del técnico acuícola.

Para este diagrama se definen tres swimlanes: *Sensor IoT / Técnico Acuícola*, *Sistema AquaGest* y *Técnico Acuícola*. El primer carril origina el dato, el segundo valida, registra y evalúa el rango de los parámetros, y el tercero recibe la alerta y ejecuta la acción correspondiente cuando se requiere intervención humana.

El flujo inicia con el envío o registro de lecturas de parámetros. Posteriormente, el sistema valida el formato de los datos y ejecuta un fork para registrar en paralelo los parámetros pH, O₂, temperatura, salinidad, turbidez y alcalinidad. Luego, un join sincroniza los registros y el sistema evalúa si algún valor se encuentra fuera del rango permitido. Si todos los valores son normales, el flujo termina con el dato almacenado. Si existe un valor fuera de rango, se genera una alerta y se notifica al técnico acuícola.

Finalmente, el diagrama debe incluir dos nodos de decisión: *¿Algún valor fuera de rango?* y *¿Requiere acción inmediata?*. También debe incorporar al menos un flujo de objeto, como *ParametroAgua fuera de rango* y *AlertaAgua*, debido a que estos objetos representan la información que pasa desde el sistema hacia la responsabilidad del técnico.

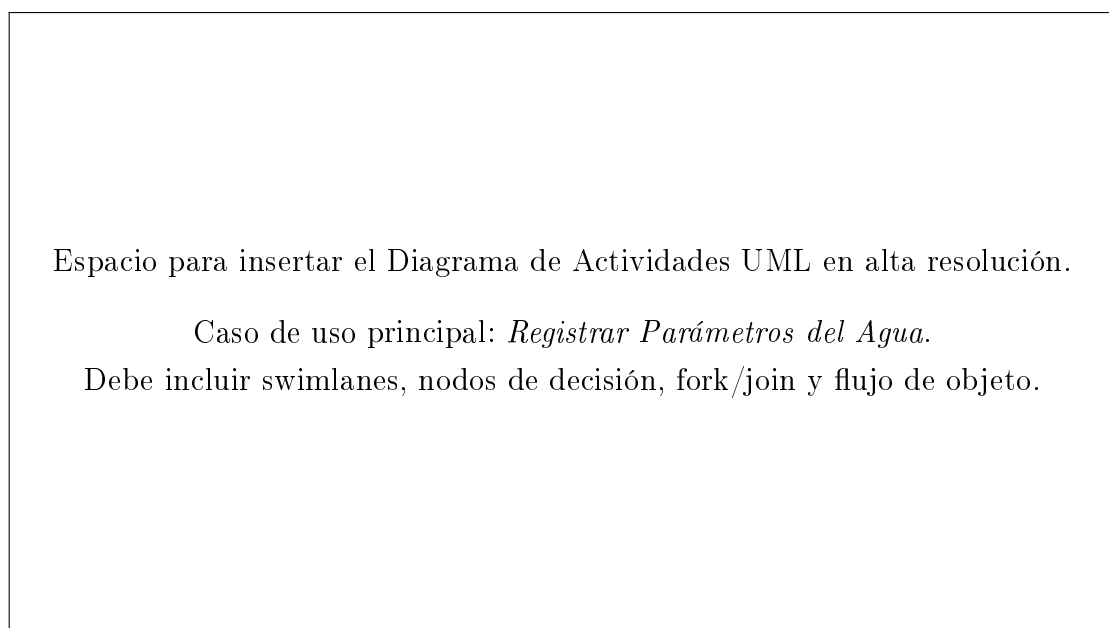


Figura 6: Diagrama de Actividades UML del caso de uso principal.

7. Modelo de comportamiento: estados y secuencia

7.1. Entidad seleccionada para el modelo de estados

La entidad seleccionada para la máquina de estados es *Estanque*. Esta selección se justifica porque el estanque es el elemento central de la operación camaronera y presenta un ciclo de vida más amplio que otras entidades del modelo. Su estado determina si se puede registrar una siembra, realizar grameos, calcular raciones de alimento, registrar cosecha o liberar el estanque para un nuevo ciclo productivo.

Aunque existen otras entidades con estados relevantes, como *AlertaAgua*, *OrdenMantenimiento*, *IncidenteSanitario* o *TareaDiaria*, estas presentan ciclos de vida más simples.

En cambio, *Estanque* posee estados de alto nivel y una fase compuesta denominada *Activo*, dentro de la cual se identifican subestados propios del proceso biológico del cultivo.

Tabla 17: Comparación de entidades candidatas para el modelo de estados.

Entidad	Estados identificables	Justificación
<i>AlertaAgua</i>	Activa, atendida.	Presenta un ciclo simple de generación y atención de alerta. No posee subestados naturales.
<i>OrdenMantenimiento</i>	Pendiente, en proceso, concluida.	Tiene un flujo operativo útil, pero limitado al control de mantenimiento.
<i>IncidenteSanitario</i>	Abierto, en tratamiento, cerrado.	Permite controlar eventos sanitarios, pero no representa el ciclo productivo completo.
<i>TareaDiaria</i>	Pendiente, completada.	Su ciclo de vida es reducido y se limita al cumplimiento de tareas asignadas.
<i>Estanque</i>	Inactivo, en preparación, activo, en cosecha.	Es la entidad con mayor riqueza de estados. Además, el estado <i>Activo</i> puede descomponerse en subestados internos: siembra reciente, en crecimiento y listo para cosecha.

Por tanto, la entidad *Estanque* se considera la más adecuada para el modelo de comportamiento, ya que articula requisitos como RF-04, RF-05, RF-06, RF-07, RF-12, RF-16, RF-22, RF-23 y RF-25. Esta relación permite evidenciar que el cambio de estado del estanque no es un dato aislado, sino una condición que afecta la ejecución de varias funcionalidades del sistema.

7.2. Identificación de estados, eventos y acciones

Los estados identificados para la entidad *Estanque* describen su evolución desde una condición inicial sin cultivo hasta su uso productivo, cosecha y liberación para un nuevo ciclo. Además, se define el estado compuesto *Activo*, debido a que durante esta fase el cultivo atraviesa subestados internos relacionados con el crecimiento del camarón.

Tabla 18: Estados identificados para la entidad Estanque.

Estado	Tipo	Descripción
<i>Inactivo</i>	Simple	El estanque no tiene cultivo activo y se encuentra disponible para iniciar preparación o permanecer fuera de operación.
<i>EnPreparacion</i>	Simple	El estanque se encuentra en limpieza, control de salinidad, acondicionamiento y preparación previa a la siembra.
<i>Activo</i>	Compuesto	Representa el ciclo biológico del cultivo. Contiene los subestados <i>SiembraReciente</i> , <i>EnCrecimiento</i> y <i>ListoParaCosecha</i> .
<i>SiembraReciente</i>	Subestado	Corresponde a las primeras semanas posteriores a la siembra, cuando todavía no existe información suficiente de grameos para estimar biomasa.
<i>EnCrecimiento</i>	Subestado	Se realizan grameos semanales y se recalcula la ración diaria de alimento conforme a la evolución del peso promedio.
<i>ListoParaCosecha</i>	Subestado	El cultivo alcanzó la talla objetivo o el módulo de IA sugirió el momento óptimo de cosecha.
<i>EnCosecha</i>	Simple	La cosecha se encuentra en proceso de registro y el estanque queda bloqueado para nuevas siembras hasta finalizar el ciclo.

La Tabla 19 presenta los eventos, condiciones y acciones que permiten transitar entre estados. Las transiciones se formulan con la estructura evento [condición] / acción, conforme a la notación esperada para una máquina de estados UML.

Tabla 19: Eventos, condiciones y acciones de la entidad Estanque.

Estado origen	Evento	Condición	Acción / estado destino
<i>Inactivo</i>	Iniciar preparación.	No aplica.	Registrar limpieza y acondicionamiento. Pasa a <i>EnPreparacion</i> .

Estado origen	Evento	Condición	Acción / estado destino
<i>EnPreparacion</i>	Registrar siembra.	Cantidad menor o igual a capacidad más sobrecarga permitida.	Crear registro de siembra e iniciar monitoreo continuo de agua. Pasa a <i>Activo / SiembraReciente</i> .
<i>SiembraReciente</i>	Registrar grameo.	Peso mayor o igual al umbral inicial.	Actualizar historial de crecimiento. Pasa a <i>EnCrecimiento</i> .
<i>EnCrecimiento</i>	Registrar grameo semanal.	Existe siembra activa.	Recalcular ración diaria. Permanece en <i>EnCrecimiento</i> .
<i>EnCrecimiento</i>	Predicción IA de punto óptimo o talla objetivo alcanzada.	Confianza mayor o igual al umbral definido.	Notificar al administrador y al técnico acuícola. Pasa a <i>ListoParaCosecha</i> .
<i>ListoParaCosecha</i>	Iniciar registro de cosecha.	Cosecha autorizada.	Bloquear nuevas siembras y detener alertas de crecimiento. Pasa a <i>EnCosecha</i> .
<i>EnCosecha</i>	Confirmar cosecha completa.	Datos de cosecha registrados.	Cerrar siembra, calcular rendimiento y liberar estanque. Pasa a <i>Inactivo</i> .
<i>Activo</i>	Mortalidad total o pérdida del ciclo.	Evento excepcional confirmado.	Registrar incidente crítico y cerrar siembra sin cosecha. Pasa a <i>Inactivo</i> .

7.3. Diagrama de Máquina de Estados UML

La máquina de estados UML representa el ciclo de vida de la entidad *Estanque*. El diagrama debe incluir siete estados en total: cuatro estados de alto nivel y tres subestados dentro del estado compuesto *Activo*. Esta estructura cumple con el criterio de la práctica, debido a que incorpora más de cinco estados, transiciones con evento, condición y acción, además de un estado compuesto con subestados internos.

Las transiciones principales son las siguientes: *Inactivo* a *EnPreparacion*; *EnPrepara-*

cion a Activo; SiembraReciente a EnCrecimiento; EnCrecimiento a ListoParaCosecha; ListoParaCosecha a EnCosecha; y EnCosecha a Inactivo. Además, se considera una transición excepcional desde cualquier subestado de Activo hacia Inactivo cuando se registra mortalidad total o pérdida del ciclo.

Tabla 20: Sintaxis de transiciones de la máquina de estados.

Transición	Sintaxis completa
<i>Inactivo</i> $\rightarrow$ <i>EnPreparacion</i>	Iniciar preparación / registrar limpieza y acondicionamiento.
<i>EnPreparacion</i> $\rightarrow$ <i>Activo</i>	Registrar siembra [cantidad $\leq$ capacidad + sobrecarga] / crear Siembra.
<i>SiembraReciente</i> $\rightarrow$ <i>EnCrecimiento</i>	Registrar grameo [peso $\geq$ umbral inicial] / actualizar historial.
<i>EnCrecimiento</i> $\rightarrow$ <i>EnCrecimiento</i>	Registrar grameo semanal / recalcular ración diaria.
<i>EnCrecimiento</i> $\rightarrow$ <i>ListoParaCosecha</i>	Predicción óptima [confianza $\geq$ umbral] o talla objetivo alcanzada / notificar.
<i>ListoParaCosecha</i> $\rightarrow$ <i>EnCosecha</i>	Iniciar registro de cosecha / bloquear nuevas siembras.
<i>EnCosecha</i> $\rightarrow$ <i>Inactivo</i>	Confirmar cosecha completa / cerrar siembra y calcular rendimiento.
<i>Activo</i> $\rightarrow$ <i>Inactivo</i>	Mortalidad total [evento excepcional] / registrar incidente crítico y cerrar siembra.

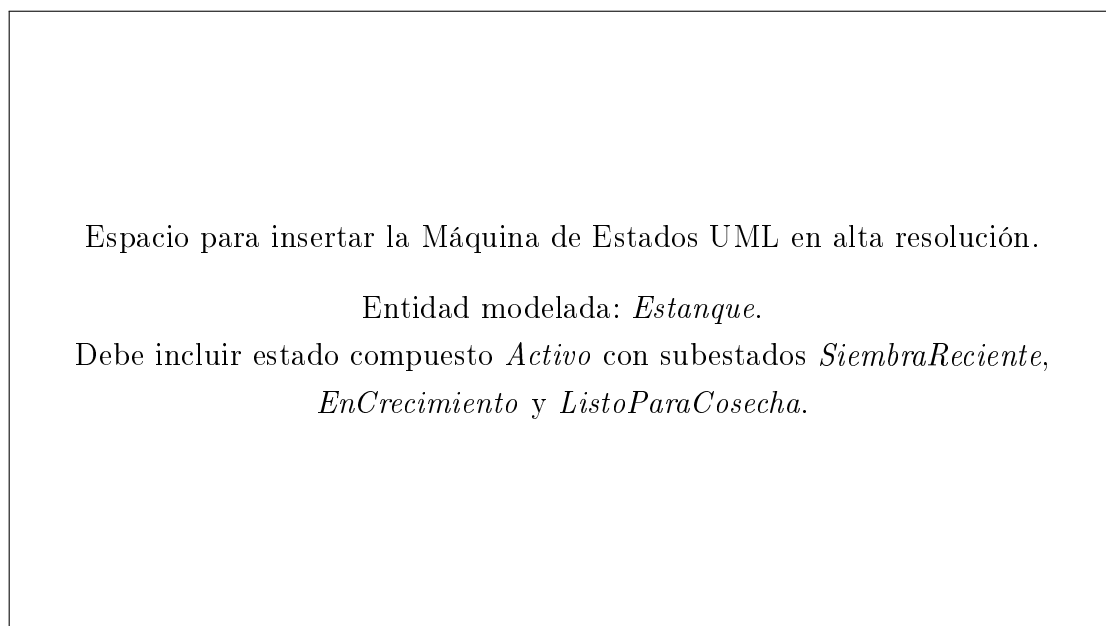


Figura 7: Máquina de Estados UML de la entidad Estanque.

7.4. Diagrama de Secuencia UML del escenario principal

El diagrama de secuencia UML se desarrolla sobre el caso de uso *CU-02 Registrar Parámetros del Agua*. Este escenario fue seleccionado porque conecta el monitoreo del cultivo, la validación de parámetros, la generación de alertas y la respuesta técnica ante valores fuera de rango. Además, se relaciona directamente con los requisitos RF-09, RF-10 y RF-11.

El diagrama debe incluir cinco líneas de vida: *Técnico Acuícola*, *Sensor IoT*, *InterfazMonitoreo*, *GestorAgua* y *AlertaAgua*. También debe incorporar un fragmento *loop*, que representa el ciclo de lectura periódica de parámetros, y un fragmento *alt*, que separa el comportamiento cuando los valores se encuentran dentro del rango permitido o fuera de este.

Tabla 21: Elementos del diagrama de secuencia del CU-02.

Elemento	Descripción
Líneas de vida	Técnico Acuícola, Sensor IoT, InterfazMonitoreo, GestorAgua y AlertaAgua.
Fragmento <i>loop</i>	Representa la lectura periódica de parámetros del agua en intervalos configurados.
Fragmento <i>alt</i>	Diferencia el escenario donde el parámetro se encuentra dentro del rango y el escenario donde se encuentra fuera de rango.

Elemento	Descripción
Mensajes síncronos	<code>registrarParametro()</code> , <code>crearAlerta()</code> , <code>registrarAccionCorrectiva()</code> y <code>cerrarAlerta()</code> .
Mensajes asíncronos o de retorno	<code>enviarLectura()</code> , <code>notificarAlerta()</code> y <code>mostrarAlerta()</code> .
Resultado esperado	El sistema registra la lectura. Si el valor está fuera de rango, crea una alerta, notifica al técnico y permite registrar la acción correctiva.

El escenario inicia cuando el Sensor IoT envía una lectura o cuando el Técnico Acuícola registra manualmente el dato en la interfaz de monitoreo. Luego, *InterfazMonitoreo* envía la información a *GestorAgua*, que valida el formato, registra el parámetro y verifica si el valor se encuentra dentro del rango permitido. Si el valor es normal, el sistema confirma el registro. Si el valor está fuera de rango, se crea una instancia de *AlertaAgua*, se notifica al técnico y se habilita el registro de la acción correctiva.

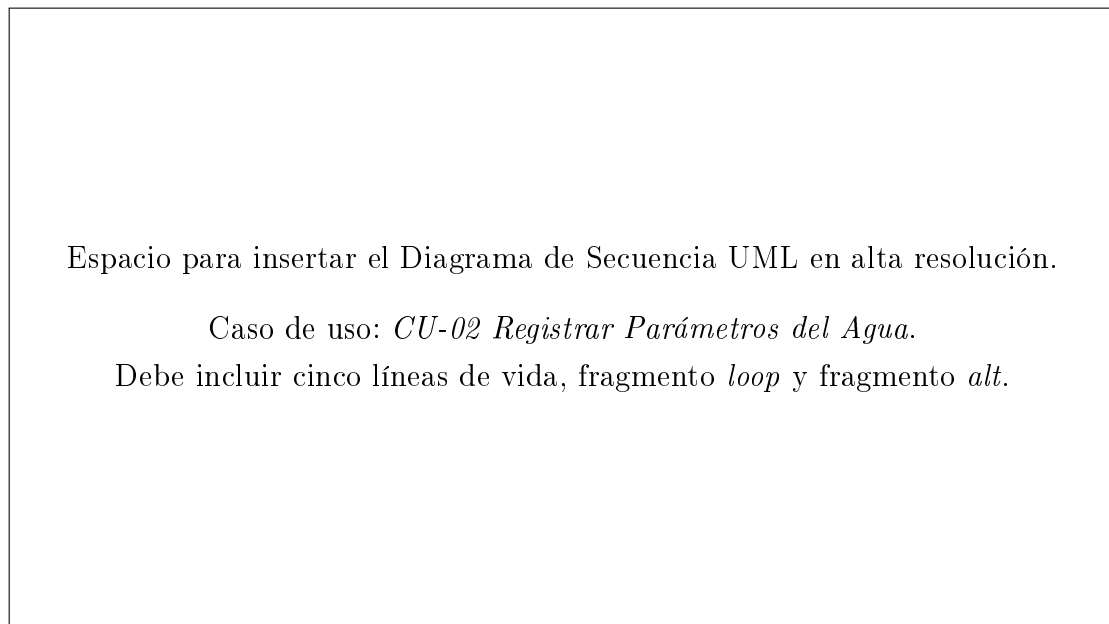


Figura 8: Diagrama de Secuencia UML del escenario principal.

8. Matriz de trazabilidad y coherencia entre modelos

8.1. Matriz RF por modelos

La Tabla 22 presenta la trazabilidad de los requisitos funcionales de AquaGest respecto a los principales modelos construidos. Se considera cobertura cuando el requisito se

encuentra representado en al menos un artefacto de análisis.

Tabla 22: Matriz de trazabilidad RF por modelos.

RF	Descripción breve	Caso de uso	Clase UML	Proceso DFD	Actividad UML	Estado UML
RF-01	Registro de estanque.	Registrar Estanque.	Estanque.	1.0	No aplica.	No aplica.
RF-02	Cálculo de capacidad máxima.	Registrar Estanque.	Estanque.	1.0	No aplica.	No aplica.
RF-03	Registro de sobrecarga controlada.	Registrar Estanque.	Estanque.	1.0	No aplica.	No aplica.
RF-04	Actualización de estado del estanque.	Actualizar Estado de Estanque.	Estanque.	1.0	No aplica.	Inactivo → EnPreparacion.
RF-05	Registro de siembra.	CU-01 Registrar Siembra.	Siembra.	1.0	No aplica.	EnPreparacion → Activo.
RF-06	Registro de grameo semanal.	Registrar Grameo.	Grameo.	1.0	No aplica.	SiembraReciente → EnCrecimiento.
RF-07	Historial de crecimiento.	Registrar Grameo.	Grameo.	1.0	No aplica.	EnCrecimiento → ListoParaCosecha.
RF-08	Alerta de crecimiento inferior.	Registrar Grameo.	Grameo.	1.0	No aplica.	No aplica.
RF-09	Registro de parámetros del agua.	CU-02 Registrar Parámetros del Agua.	ParametroAgua.	2.0	Aplica.	No aplica.
RF-10	Alerta por parámetro fuera de rango.	Gestionar Alerta de Agua.	ParametroAgua, AlertaAgua.	2.0	Aplica.	No aplica.
RF-11	Registro de acción correctiva.	Gestionar Alerta de Agua.	AlertaAgua.	2.0	Aplica.	No aplica.
RF-12	Cálculo de ración diaria.	Calcular Ración Diaria.	RegistroAlimentacion.	3.0	No aplica.	Self-loop en EnCrecimiento.
RF-13	Registro de consumo de alimento.	CU-03 Registrar Alimentación Diaria.	RegistroAlimentacion.	3.0	No aplica.	No aplica.
RF-14	Informe de eficiencia alimenticia FCR.	Calcular Ración Diaria.	RegistroAlimentacion.	3.0	No aplica.	No aplica.
RF-15	Registro de incidente sanitario.	Registrar Incidente Sanitario.	IncidenteSanitario.	4.0	No aplica.	No aplica.
RF-16	Alerta de mortalidad anormal.	Registrar Incidente Sanitario.	IncidenteSanitario.	4.0	No aplica.	Activo → Inactivo.
RF-17	Registro de empleados.	Registrar Empleado.	Empleado.	5.0	No aplica.	No aplica.
RF-18	Asignación y control de tareas.	Asignar Tarea Diaria.	Sin clase definida.	5.0	No aplica.	No aplica.
RF-19	Gestión de inventario de insumos.	Gestionar Inventario de Insumos.	Insumo.	6.0	No aplica.	No aplica.

RF	Descripción breve	Caso de uso	Clase UML	Proceso DFD	Actividad UML	Estado UML
RF-20	Alerta de stock mínimo.	Gestionar Inventario de Insumos.	Insumo.	6.0	No aplica.	No aplica.
RF-21	Registro de órdenes de mantenimiento.	Registrar Orden de Mantenimiento.	Sin clase definida.	6.0	No aplica.	No aplica.
RF-22	Registro de cosecha.	CU-04 Registrar Cosecha.	Cosecha.	7.0	No aplica.	ListoParaCosecha → EnCosecha → Inactivo.
RF-23	Cálculo de costo de producción.	Calcular Costo de Producción.	Cosecha.	7.0	No aplica.	EnCosecha → Inactivo.
RF-24	Reportes comparativos de producción.	Generar Reporte Comparativo.	Sin clase específica; reporte de consulta.	7.0	No aplica.	No aplica.
RF-25	Predicción de momento óptimo de cosecha.	CU-05 Predicción de Cosecha Óptima.	Sin clase definida.	8.0	No aplica.	EnCrecimiento → ListoParaCosecha.

Tabla 23: Resumen de cobertura de modelos.

Modelo	RF cubiertos	Cobertura
Casos de uso	25 / 25	100 %
Clases UML	22 / 25	88 %
Procesos DFD Nivel 1	25 / 25	100 %
Diagrama de Actividades UML	3 / 25	12 %
Máquina de Estados UML	9 / 25	36 %

Las columnas de actividades UML y estados UML presentan menor cobertura porque la práctica profundiza en un único caso de uso principal y una única entidad con ciclo de vida relevante. Esta limitación corresponde al alcance definido para la PE3 y no implica necesariamente ausencia de requisitos, siempre que se documente la decisión metodológica.

8.2. Requisitos sin cobertura y modelos sin requisito asociado

La revisión de cobertura permite distinguir entre gaps reales y decisiones de diseño. Un gap ocurre cuando un requisito no tiene soporte suficiente en los modelos construidos. En cambio, el gold plating aparece cuando se incorpora un elemento de diseño que no está respaldado directamente por un requisito funcional.

Tabla 24: Gaps identificados en la trazabilidad de modelos.

RF	Gap identificado	Justificación y acción recomendada
RF-18	No existe clase <i>TareaDiaria</i> en el diagrama de clases.	El requisito exige asignar tareas y controlar su cumplimiento; sin una clase que almacene estado, responsable y fecha, la funcionalidad queda incompleta. Se recomienda agregar <i>TareaDiaria</i> .
RF-21	No existe clase <i>OrdenMantenimiento</i> .	El requisito requiere registrar órdenes, estados e historial de mantenimiento. Se recomienda incorporar una clase o entidad <i>OrdenMantenimiento</i> .

RF	Gap identificado	Justificación y acción recomendada
RF-25	No existe clase <i>PrediccionIA</i> .	El requisito requiere gestionar predicciones, confianza y datos asociados. Se recomienda agregar <i>PrediccionIA</i> para almacenar cada resultado generado por el módulo de IA.

Tabla 25: Elementos identificados como posible gold plating.

Elemento del modelo	RF que debería justificarlo	Evaluación
<i>AsignacionEstanque</i>	No existe un RF que solicite explícitamente asignar empleados a estanques.	Gold plating claro. La entidad fue introducida para resolver una segunda relación M:N y debe documentarse como decisión de diseño del equipo.
Fork/join de seis registros paralelos en actividades UML.	RF-09 exige registrar mediciones, pero no exige concurrencia.	Gold plating leve. Es una interpretación razonable por el uso de sensores, pero debe documentarse como supuesto técnico.
Subestados de <i>Activo</i> .	RF-04 solo define estados generales del estanque.	Zona gris. Los subestados se justifican parcialmente por RF-06, RF-07 y RF-08; deben presentarse como refinamiento técnico, no como nuevo requisito.

A partir de esta revisión se concluye que no existen requisitos completamente descubiertos, ya que todos los RF tienen caso de uso y proceso DFD asociado. Sin embargo, sí se identifican gaps estructurales en RF-18, RF-21 y RF-25, debido a que carecen de clases de soporte en el modelo UML. Estos hallazgos deben corregirse antes de una versión final del diseño.

8.3. Verificaciones cruzadas entre modelos

Las verificaciones cruzadas permiten comprobar si los modelos mantienen coherencia entre sí. Esta revisión permite detectar inconsistencias entre datos, procesos y comportamiento antes de avanzar hacia fases posteriores de diseño o implementación.

Tabla 26: Verificaciones cruzadas entre perspectivas de modelado.

Cruce	Elemento revisado	Resultado de coherencia
ER – Clases UML	Entidades principales y clases del dominio.	Las entidades del ER se trasladan a clases UML; sin embargo, se identifican gaps en <i>TareaDiaria</i> , <i>OrdenMantenimiento</i> y <i>PrediccionIA</i> .
Clases UML – DFD	Procesos funcionales y clases que almacenan datos.	Los procesos DFD tienen soporte en entidades y clases principales. No obstante, algunos procesos como tareas, mantenimiento e IA requieren clases adicionales.
Estados UML – Actividades UML	Estados del estanque y flujo de parámetros del agua.	El estado del estanque condiciona operaciones como siembra, alimentación y cosecha. El diagrama de actividades complementa el comportamiento del monitoreo de agua.
DFD – Matriz de RF	Subprocesos DFD frente a requisitos funcionales.	Los 25 requisitos funcionales se encuentran cubiertos por al menos un proceso DFD Nivel 1.
Secuencia UML – Actividades UML	CU-02 Registrar Parámetros del Agua.	Ambos modelos representan el mismo proceso desde perspectivas distintas: actividades muestra flujo general y secuencia muestra interacción temporal entre objetos.

8.4. Tabla comparativa de perspectivas de modelado

La comparación entre modelos permite evidenciar que cada perspectiva cubre una dimensión distinta del sistema. El modelo estructural permite analizar datos y relaciones; el modelo funcional permite comprender procesos y flujos; y el modelo de comportamiento permite representar eventos, estados y secuencias temporales.

Tabla 27: Comparación de modelos estructurales, funcionales y de comportamiento.

Criterio	Modelo estructural ER / Clases UML	Modelo funcional DFD / Actividades UML	Modelo de comportamiento Estados / Secuencia UML
Perspectiva cubierta	Representa qué información persiste el sistema y cómo se relaciona. Responde a la pregunta: ¿qué existe?	Representa cómo fluye la información entre actores, procesos y almacenes. Responde a la pregunta: ¿qué se hace y con qué datos?	Representa cómo cambian los estados y cómo interactúan los objetos en un escenario. Responde a la pregunta: ¿cuándo ocurre y en qué orden?
Notación utilizada	Crow's Foot para ER y UML de clases con visibilidades, atributos y operaciones.	DFD Nivel 0, DFD Nivel 1, DFD esencial y UML de actividades con swimlanes, fork/join y flujo de objeto.	Máquina de estados UML con estado compuesto y diagrama de secuencia UML con fragmentos alt y loop.
RF que ayuda a descubrir o depurar	Permite detectar requisitos sin estructura de persistencia. En AquaGest permitió identificar gaps para RF-18, RF-21 y RF-25.	Permite detectar responsabilidades funcionales y flujos mal delimitados. En AquaGest ayudó a ubicar procesos como FCR, inventario, reportes e IA.	Permite detectar reglas temporales, transiciones implícitas y caminos alternos. En AquaGest ayudó a modelar cosecha, mortalidad y acciones correctivas.
Limitaciones	No representa orden temporal, eventos ni condiciones de ejecución. Una relación correcta no explica cuándo ocurre un proceso.	No representa el ciclo de vida completo de una entidad específica ni diferencia todos los mensajes síncronos o asíncronos.	Tiene alto costo de mantenimiento si se modelan todas las entidades y todos los casos de uso con el mismo nivel de detalle.
Ejemplo aplicado a AquaGest	El modelo permitió separar <i>Estanque</i> , <i>Siembra</i> , <i>Grameo</i> , <i>Insumo</i> y <i>RegistroAlimentacion</i> , evitando redundancias.	El DFD Nivel 1 permitió organizar los procesos de siembra, agua, alimentación, sanidad, inventario, cosecha y predicción IA.	La máquina de estados permitió representar que el estanque pasa de preparación a activo, luego a cosecha y finalmente a inactivo.

En conclusión, los tres modelos son complementarios y no sustituibles entre sí. El modelo estructural sin el funcional no explica quién utiliza los datos ni para qué; el modelo funcional sin el de comportamiento no explica condiciones, eventos o cambios de estado; y el modelo de comportamiento sin el estructural no tiene entidades ni atributos sobre los cuales aplicar sus transiciones. Por ello, la verificación cruzada entre modelos permite diferenciar gaps reales, decisiones de diseño y posibles casos de gold plating.

9. Conclusiones

9.1. Conclusión 1

9.2. Conclusión 2

9.3. Conclusión 3

9.4. Conclusión 4

9.5. Conclusión 5

10. Referencias bibliográficas

Referencias

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and software engineering – Life cycle processes – Requirements engineering*. Geneva, Switzerland: ISO, 2018.
- [2] ISO/IEC/IEEE, *ISO/IEC/IEEE 15288:2023 Systems and software engineering – System life cycle processes*. Geneva, Switzerland: ISO, 2023.
- [3] ISO/IEC/IEEE, *ISO/IEC/IEEE 12207:2017 Systems and software engineering – Software life cycle processes*. Geneva, Switzerland: ISO, 2017.
- [4] ISO/IEC, *ISO/IEC 25010:2023 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*. Geneva, Switzerland: ISO, 2023.
- [5] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Cham, Switzerland: Springer, 2025.

- [6] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. Los Alamitos, CA, USA: IEEE Computer Society, 2024.
- [7] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th ed.*. Newtown Square, PA, USA: Project Management Institute, 2021.
- [8] International Requirements Engineering Board, *IREB Certified Professional for Requirements Engineering – Foundation Level – Syllabus, Version 3.1*. Karlsruhe, Germany: IREB, 2022.
- [9] L. A. Maciaszek, *Requirements Analysis and System Design*, 3rd ed. Harlow, England: Pearson Education, 2007.
- [10] D. Harel, “Statecharts: A visual formalism for complex systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987.
- [11] E. Hull, K. Jackson, and J. Dick, *Requirements Engineering*, 3rd ed. London, England: Springer, 2011.
- [12] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Needham, MA, USA: OMG, 2017.
- [13] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [14] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, “Easy Approach to Requirements Syntax (EARS),” in *Proc. 17th IEEE International Requirements Engineering Conference*, Atlanta, GA, USA, 2009, pp. 317–322.

Referencias

- [1] [Completar referencia IEEE].
- [2] [Completar referencia IEEE].
- [3] [Completar referencia IEEE].

A. Tabla completa de atributos de RF y RNF

B. Registro de defectos del SRS con correcciones aplicadas

C. Exportaciones originales de diagramas en alta resolución

C.1. Diagrama ER

C.2. Diagrama de Clases UML

C.3. DFD Nivel 0

C.4. DFD Nivel 1

C.5. DFD esencial

C.6. Diagrama de Actividades UML

C.7. Máquina de Estados UML

C.8. Diagrama de Secuencia UML

D. Referencias IEEE y declaración de uso de IA

D.1. Captura o evidencia del gestor bibliográfico

D.2. Declaración de uso de herramientas de IA

E. Declaración individual de aporte de cada integrante

E.1. Integrante 1

E.2. Integrante 2

E.3. Integrante 3

E.4. Integrante 4